[NLP & Text Mining]

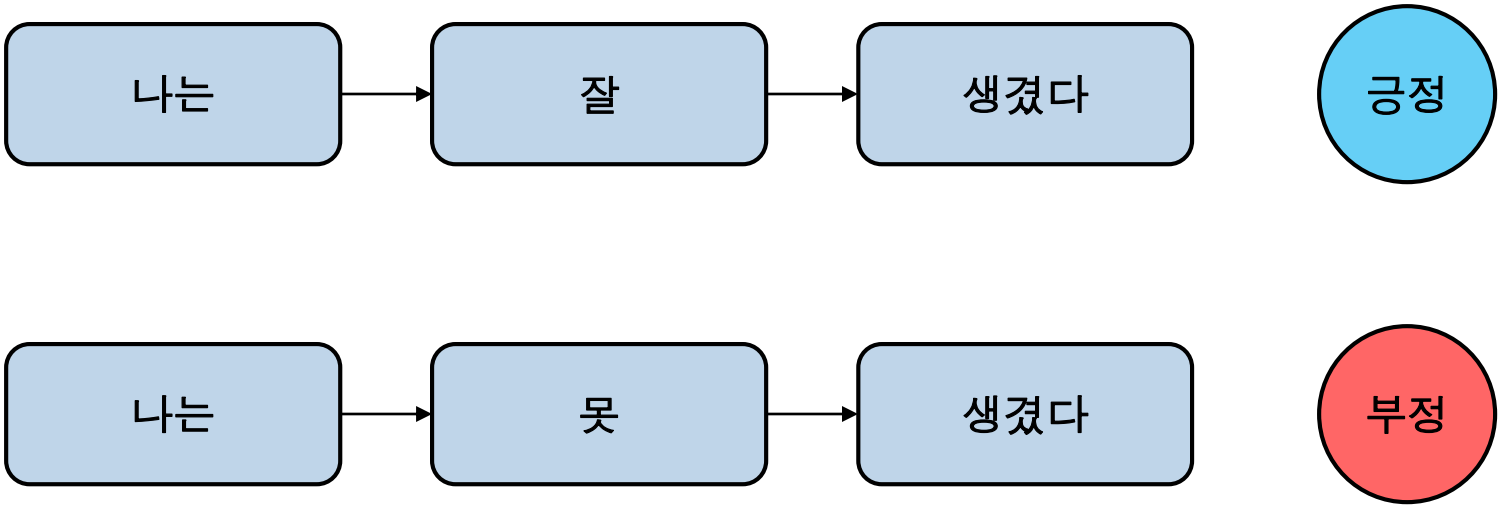
RNN

박성호

컴퓨터공학부

텍스트

- 문장은 단어로 구성되며 단어의 순서가 중요
- 감성분석



감성분석

- 기존에는 document term matrix를 구성하여 분류 모델 구축
- 문장의 순서를 고려하지 못함

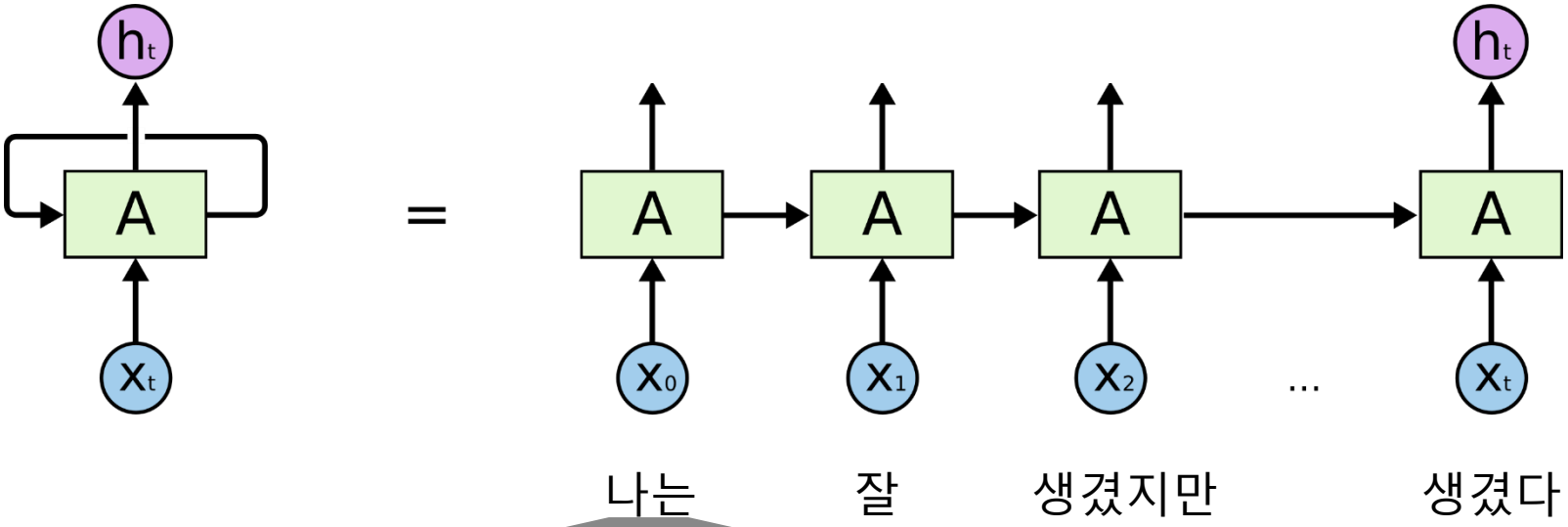
모든 단어

	나는	잘	못	생겼다	Y
나는 잘 생겼다	1	1	0	1	긍정
나는 못 생겼다	1	0	1	1	부정

	나는	잘	못	생겼지만	너는	생겼다
나는 잘 생겼지만 너는 못 생겼다	1	1	1	1	1	1
너는 잘 생겼지만 나는 못 생겼다	1	1	1	1	1	1

RNN

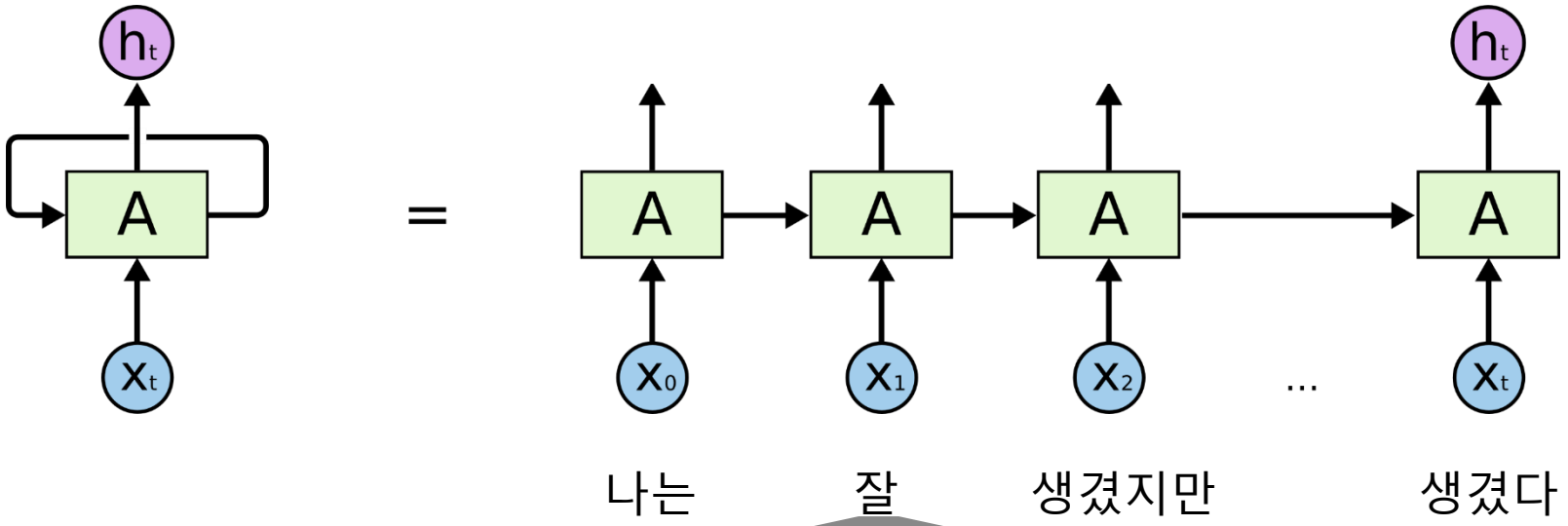
- 순서를 고려할 수 있는 RNN 모델
- 현재의 정보를 미래에 반영



나는	잘	못	생겼지만	너는	생겼다
1	0	0	0	0	0

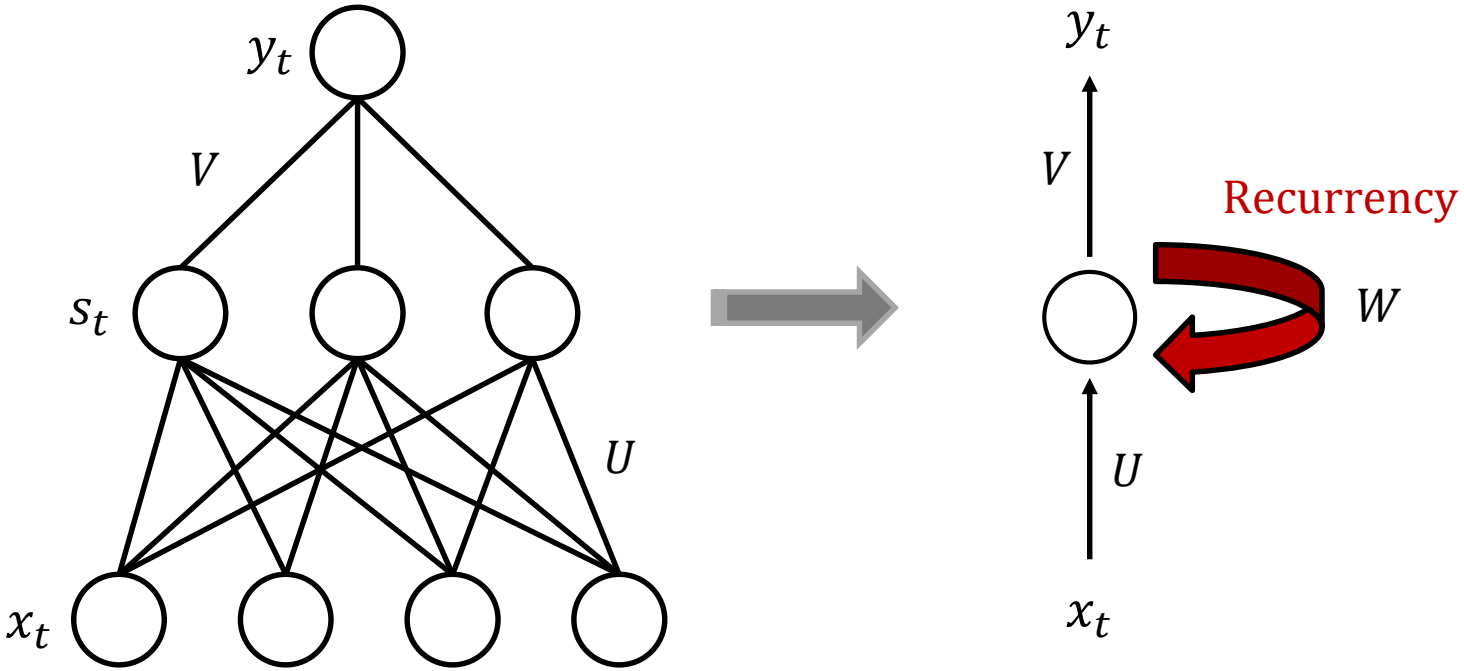
RNN (Recurrent Neural Networks)

- 순서를 고려할 수 있는 RNN 모델
- 현재의 정보를 미래에 반영



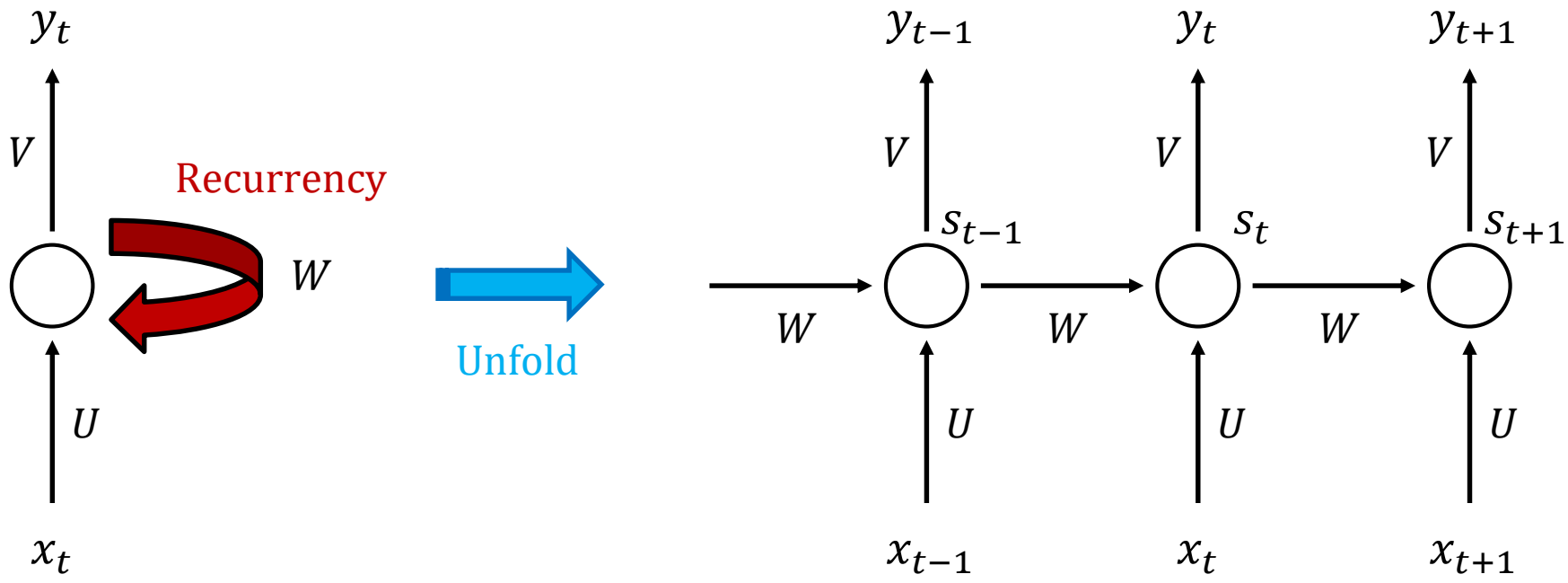
나는	잘	못	생겼지만	너는	생겼다
0	1	0	0	0	0

RNN



RNN

Recurrent neural networks share the same parameters (U, V, W) across all time steps.

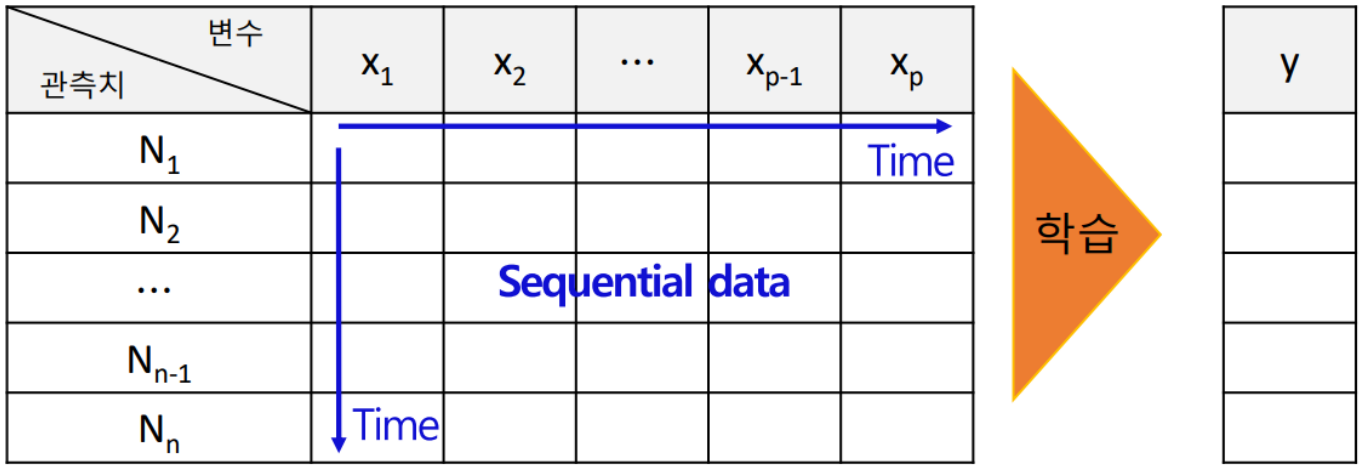


RNN

❖ 순환 신경망(recurrent neural network, RNN)

$$f(\cdot) = \text{RNN Model}$$

설명 변수(X) $f(X) = y$ 반응 변수(y)



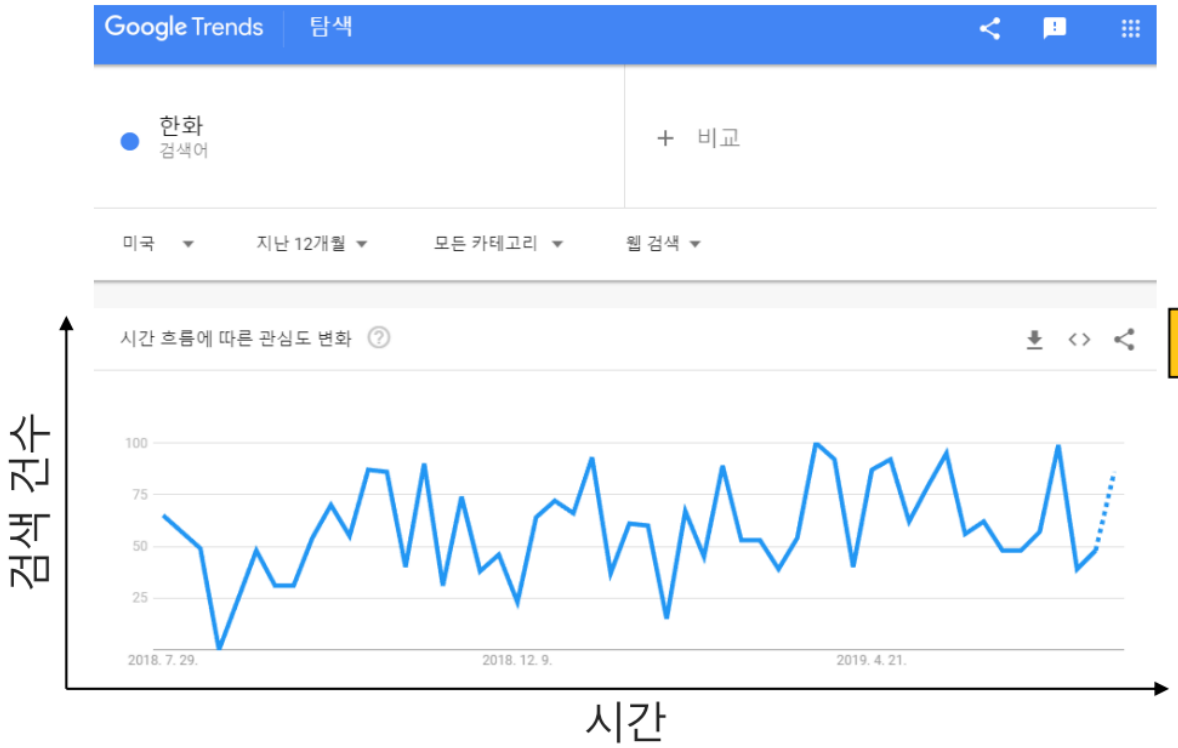
순차 데이터(sequential data) 모델링을 위한 인공 신경망(neural network) 모델

RNN

❖ 순차 데이터(sequential data)

- **변수** 간 or **관측치** 간 순서가 있는 데이터

구글 트렌드 데이터: 검색어 '한화'



Raw data

시간	검색 건수
T=1	
…	…
T=10	
T=11	
…	…
T=20	
T=21	
…	…
T=30	

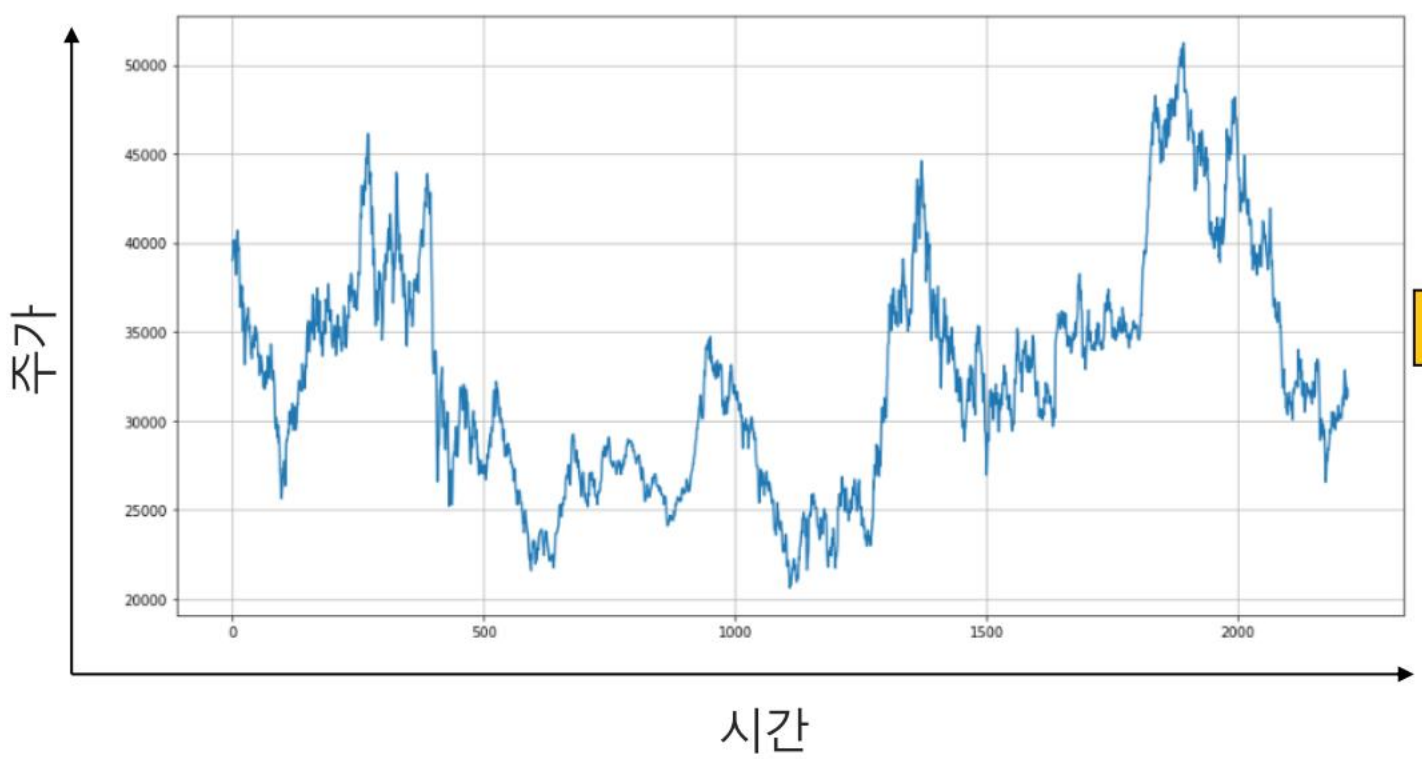
Time

RNN

❖ 순차 데이터(sequential data)

- **변수** 간 or **관측치** 간 순서가 있는 데이터

한화 주가 데이터 (2010년 ~ 2018년)



시간	주가
2010.1.1	…
…	…
2010.12. 31	…
…	…
…	…
…	…
2018.1.1	…
…	…
2018.12.31	…

Time

RNN

❖ 순차 데이터(sequential data)

- **변수** 간 or **관측치** 간 순서가 있는 데이터

텍스트 데이터-고객 여행상품 구매 후기 데이터

	리뷰	평점
0	excellent service, great price, an...	5
1	Comfortable seating with adequate ...	5
2	I like Flix Bus. It seems like an ...	4
3	Very good service, clean bus, big,...	4
4	Great first trip! Great price, cle...	5
5	It was a very bad trip. The bus de...	1
6	Im first time by flixbus, it's nic...	4
7	Good way to travel	4
8	Very fast, very cheap and rather c...	4
9	Booked bus from Bayreuth in German...	1

시간

RNN

❖ 순차 데이터(sequential data)

- **변수** 간 or **관측치** 간 순서가 있는 데이터

텍스트 데이터-고객 제품 구매 후기 데이터

	리뷰	평점
0	excellent service, great price, an...	5
1	Comfortable seating with adequate ...	5
2	I like Flix Bus. It seems like an ...	4
3	Very good service, clean bus, big,...	4
4	Great first trip! Great price, cle...	5
5	It was a very bad trip. The bus de...	1
6	Im first time by flixbus, it's nic...	4
7	Good way to travel	4
8	Very fast, very cheap and rather c...	4
9	Booked bus from Bayreuth in German...	1

Time

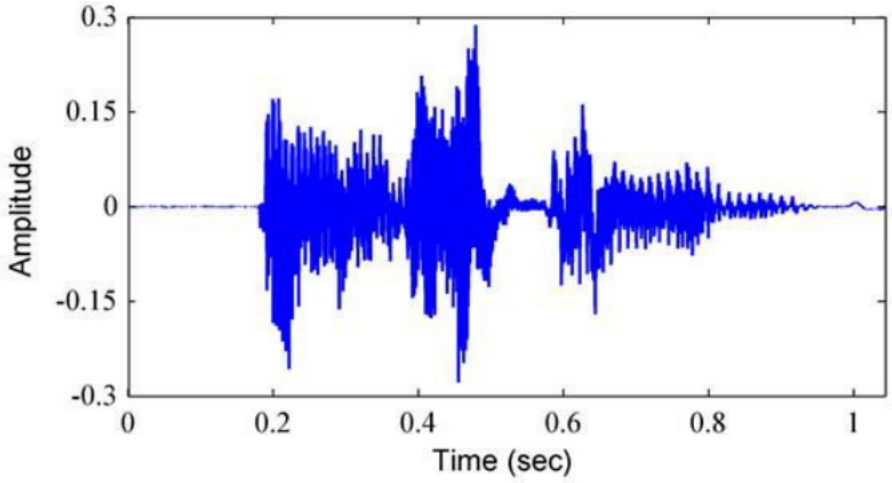
	T=1	T=2	…	T=9	T=10	별점
리뷰$_1$	문장 1	문장 2	…	문장 9	문장 10	Y_1
리뷰$_2$	문장 1	문장 2	…	문장 9	문장 10	Y_2
…	…	…	…	…	…	…
리뷰$_n$	문장 1	문장 2	…	문장 9	문장 10	y_n

RNN

❖ 순차 데이터(sequential data)

- **변수** 간 or **관측치** 간 순서가 있는 데이터

음성 데이터 - speech recognition



음소 단위

아 ㄴ 녕 ㅎ ㅏ ㅅ ㅔ ㅇ ㅛ

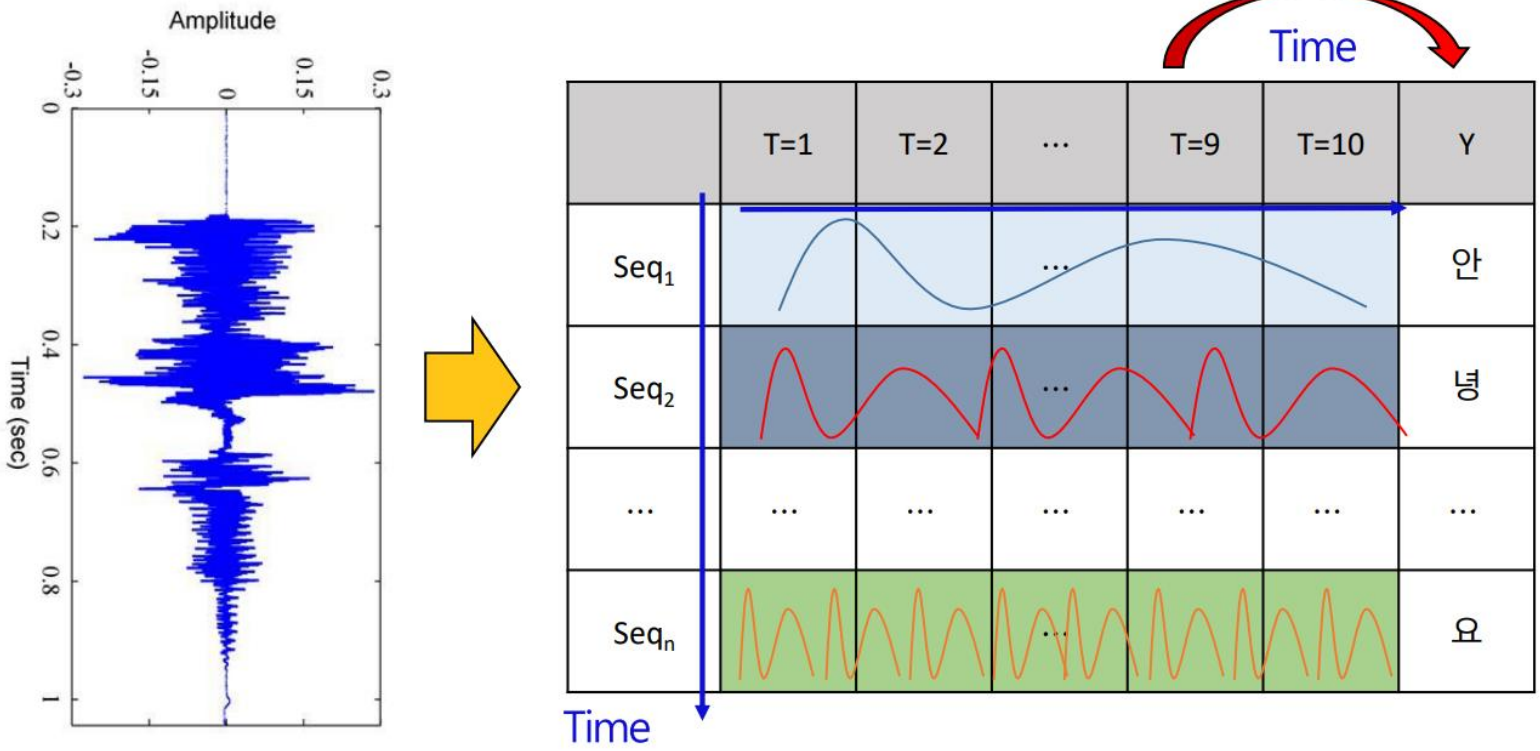
RNN

❖ 순차 데이터(sequential data)

- **변수** 간 or **관측치** 간 순서가 있는 데이터

음소 단위

$$f(\cdot) = \text{RNN Model}$$



RNN

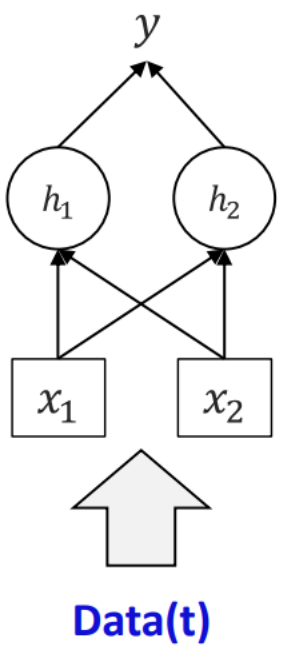
❖ 신경망(NN) vs 순환 신경망(RNN)

$$f(\cdot) = \text{Recurrent Neural Network}$$

반응 변수

은닉층

설명 변수



→ Y를 잘 예측하는 방향으로.

→ 더 좋은 특징을 추출

Hidden state

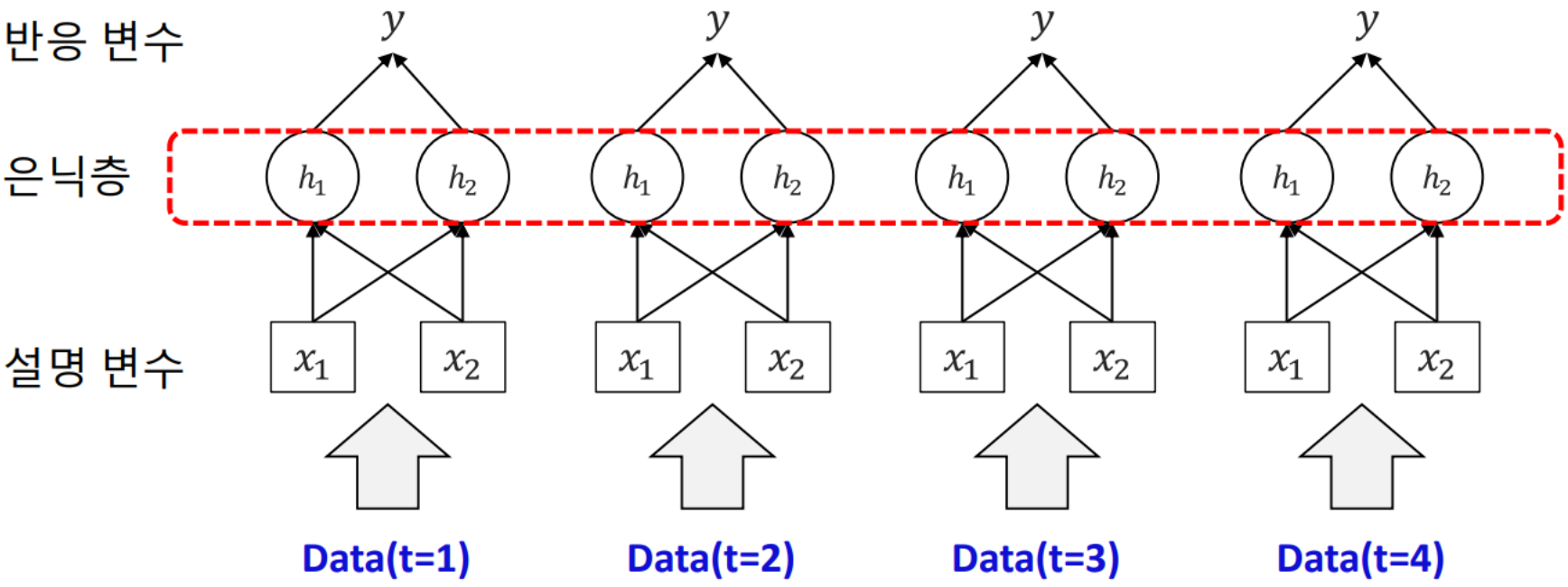
→ 변수 2개를 조합

RNN

❖ 신경망(NN) vs 순환 신경망(RNN)

$$f(\cdot) = \text{Recurrent Neural Network}$$

시간을 반영하는 것은 Hidden layer, 어떻게?

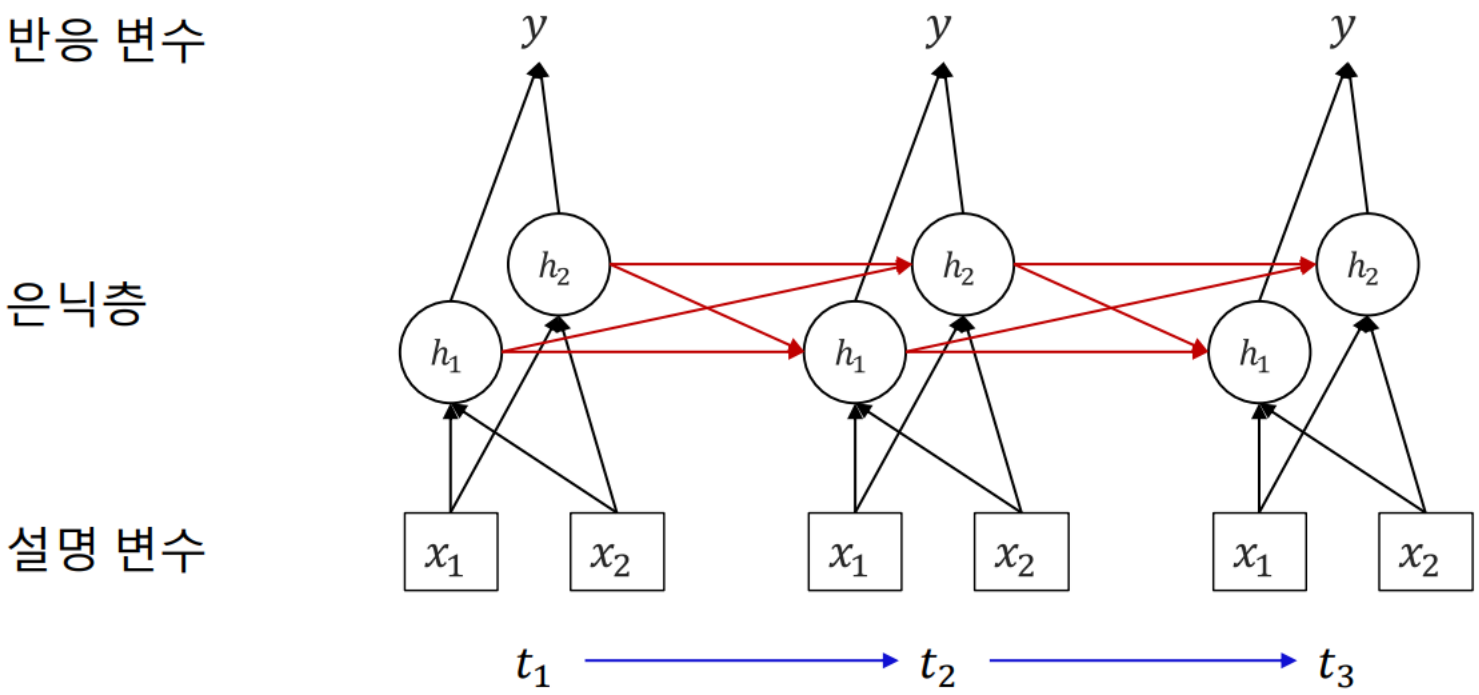


RNN

❖ 신경망(NN) vs 순환 신경망(RNN)

Hidden state를 통해 정보를 전달

$$f(\cdot) = \text{Recurrent Neural Network}$$

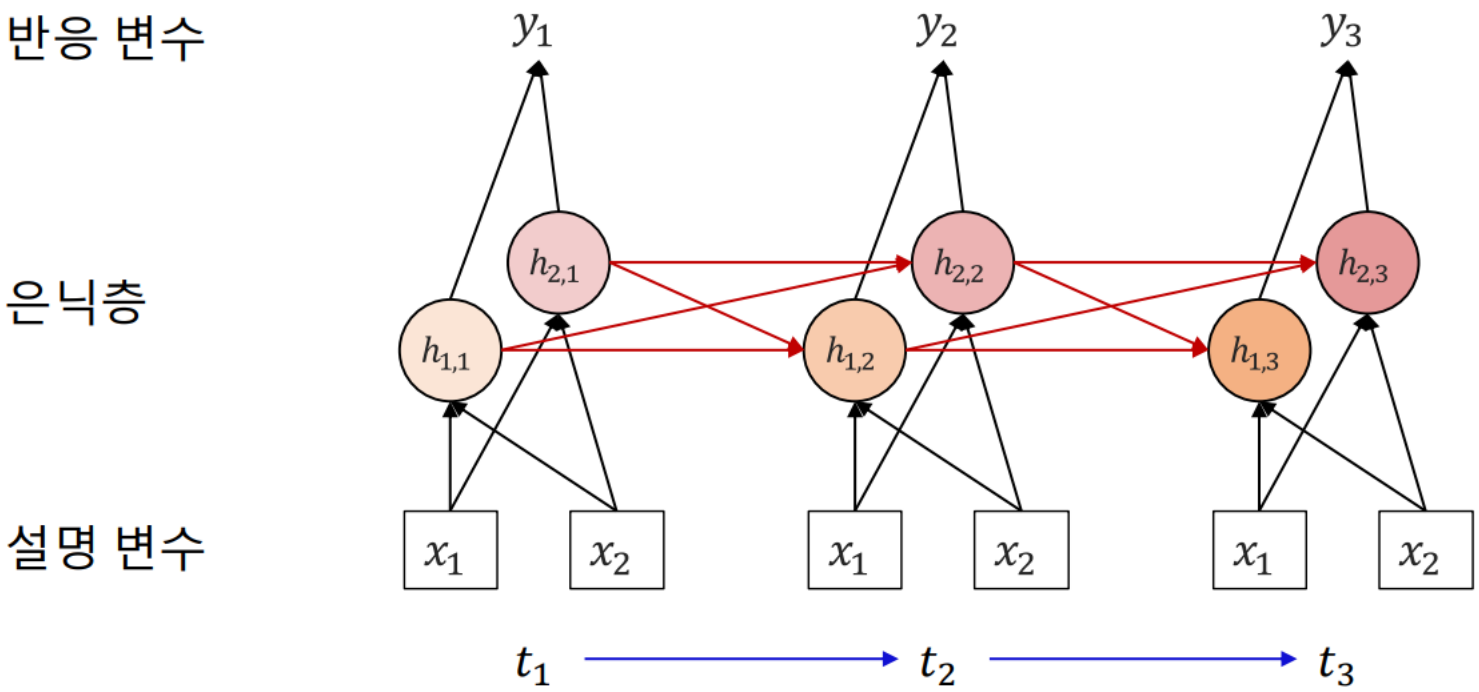


RNN

❖ 순환 신경망(RNN)

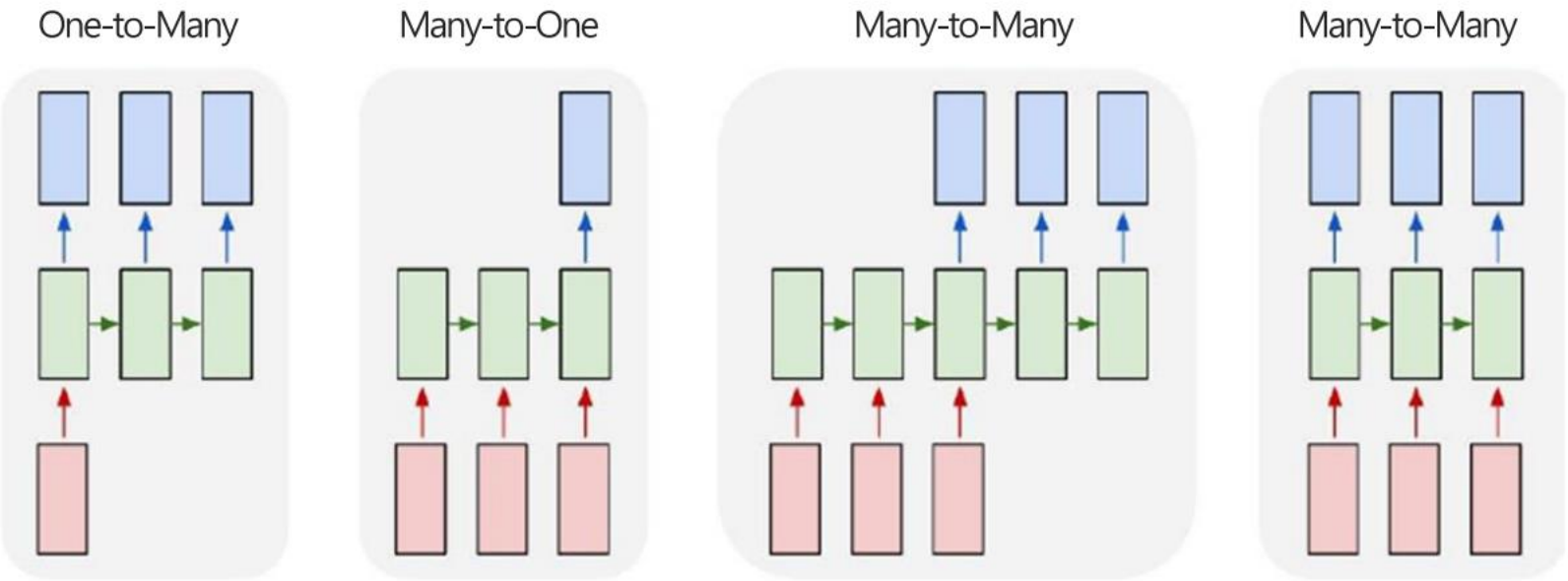
t 시점 x 데이터가 input되었을 때, t 시점 y를 예측

$$f(\cdot) = \text{Recurrent Neural Network}$$



RNN

❖ RNN 구조



이미지 캡셔닝
input: 이미지
output: 문장

문장분류
감성 분류

기계번역
seq-to-seq과 관련
문장을 모두 입력 후
번역

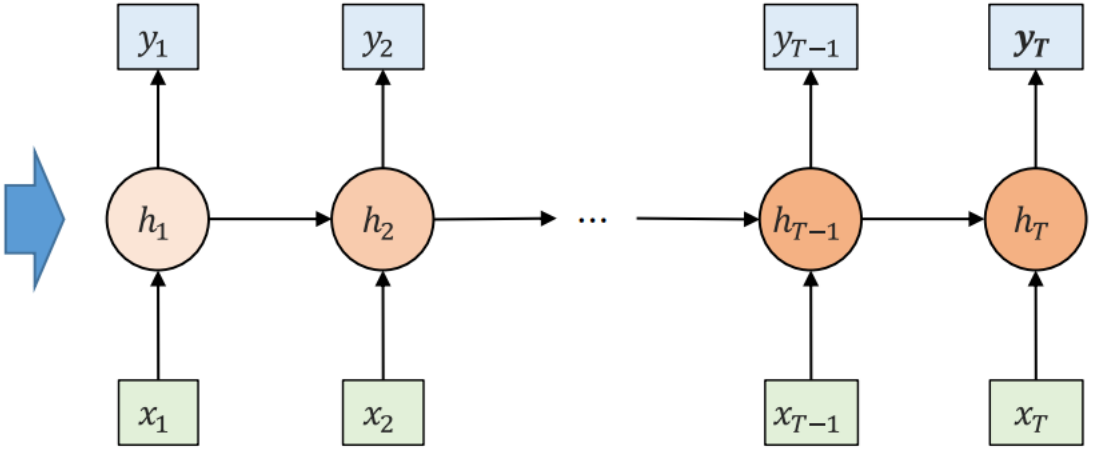
초기 기계번역
현재는 동영상
데이터 분석

RNN

❖ RNN 구조

Many to Many RNN

Task: Time-series Forecasting

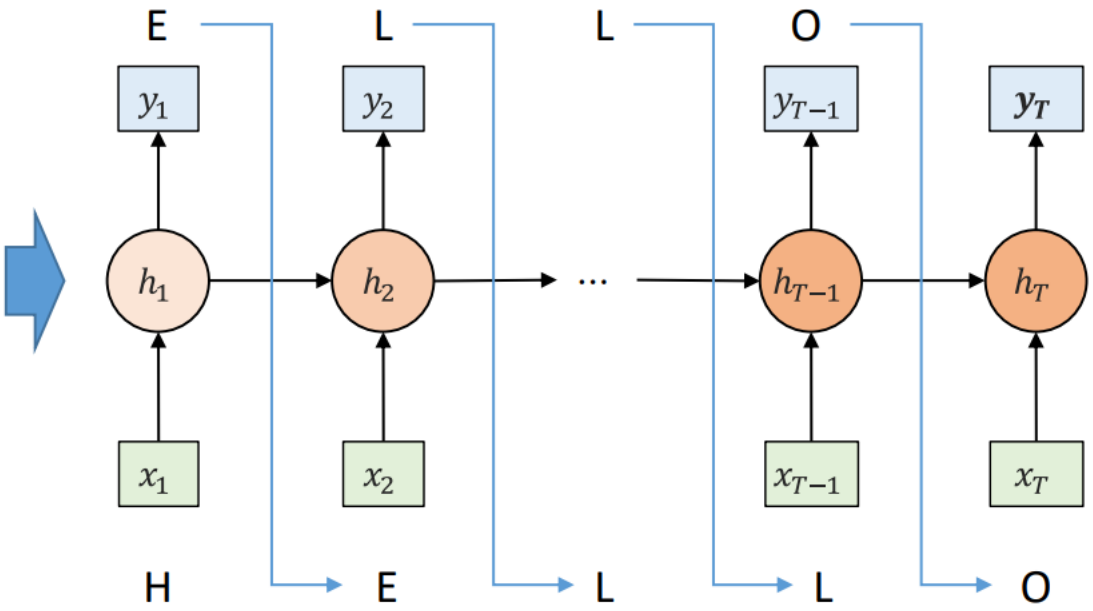


RNN

❖ RNN 구조

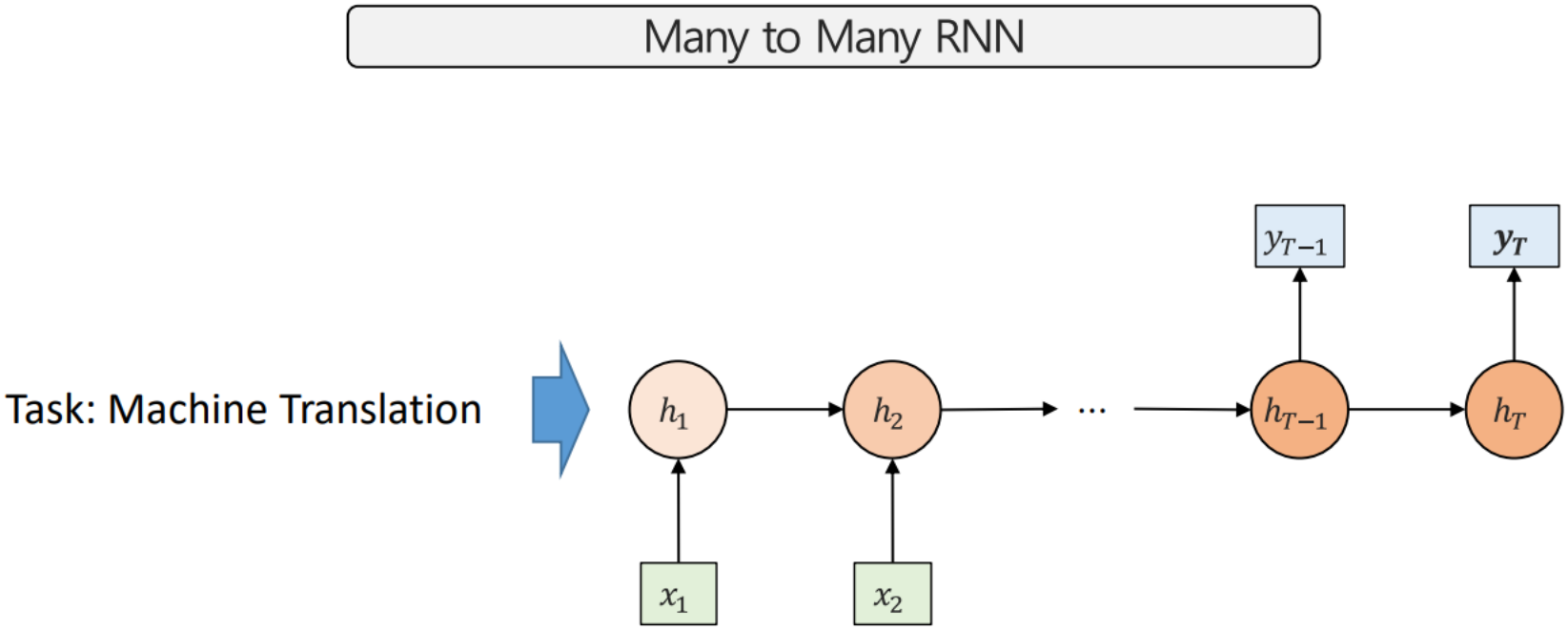
Many to Many RNN

Task: Language modeling



RNN

❖ RNN 구조


Many to Many RNN
Task: Machine Translation
h_1
h_2
h_{T-1}
h_T
x_1
x_2
y_{T-1}
y_T

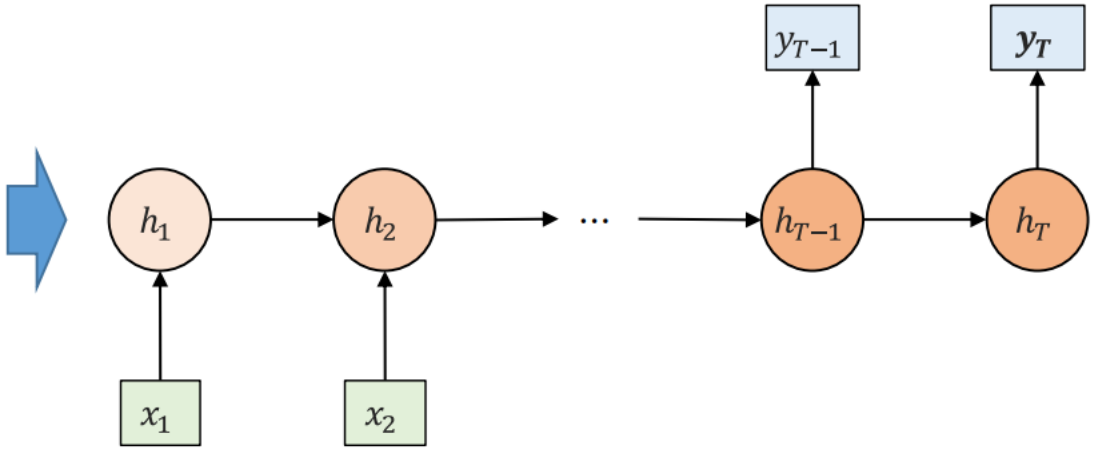
RNN

❖ RNN 구조

Many to Many RNN

(한국어) 안녕하세요 조 박사님, 한화입니다.

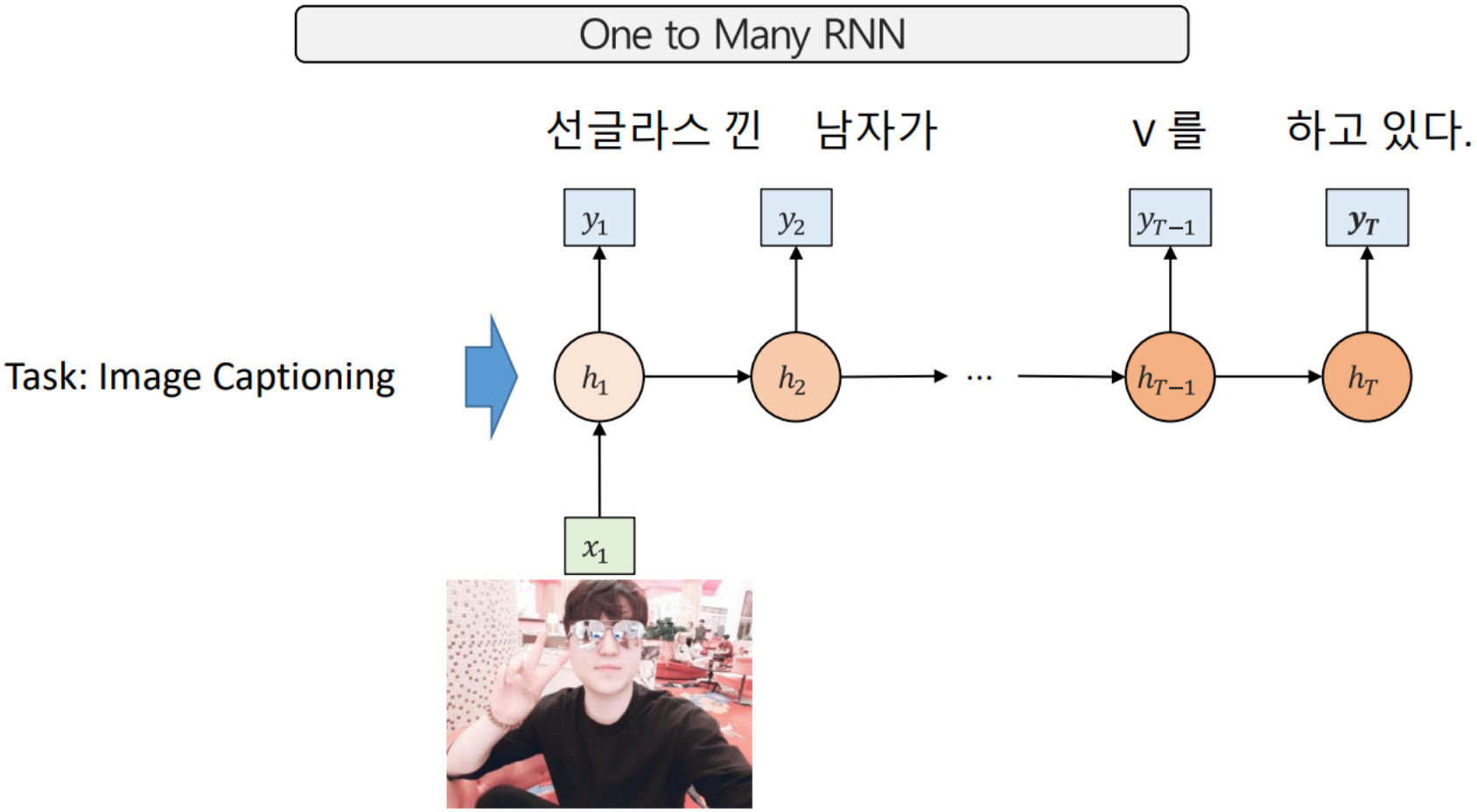
Task: Machine Translation



(English) Hello Dr. Cho, This is Hanhwa.

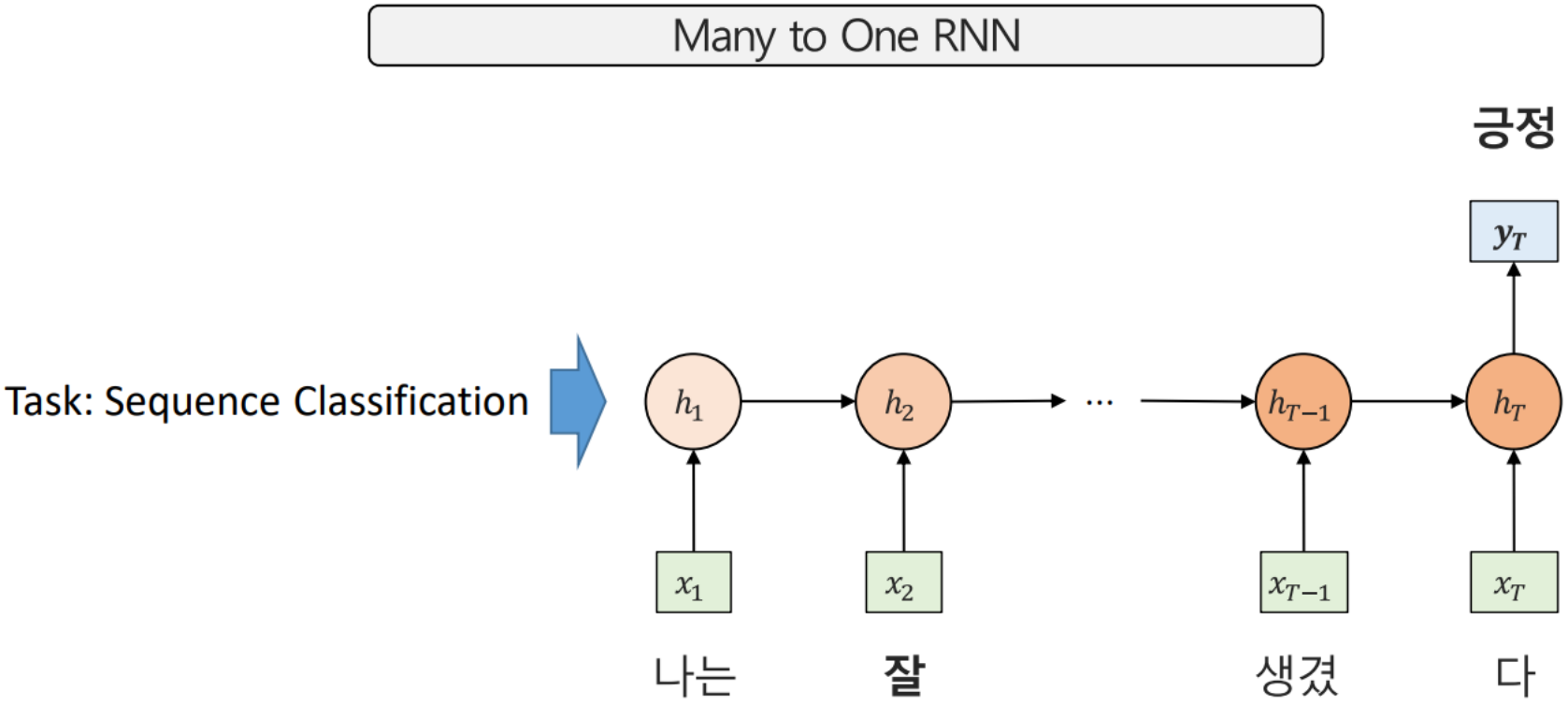
RNN

❖ RNN 구조


One to Many RNN
선글라스 낀 남자가 V 를 하고 있다.
Task: Image Captioning
y_1
y_2
y_{T-1}
y_T
h_1
h_2
...
h_{T-1}
h_T
x_1

RNN

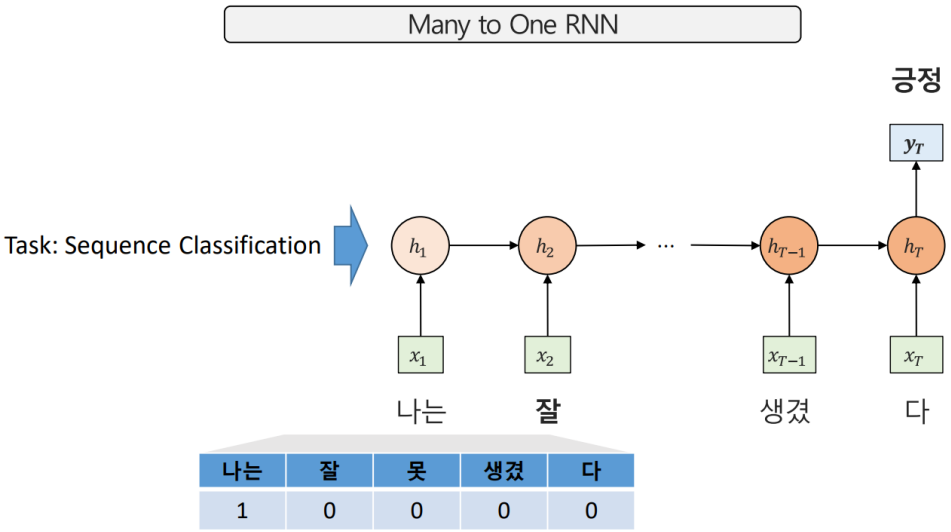
❖ RNN 구조



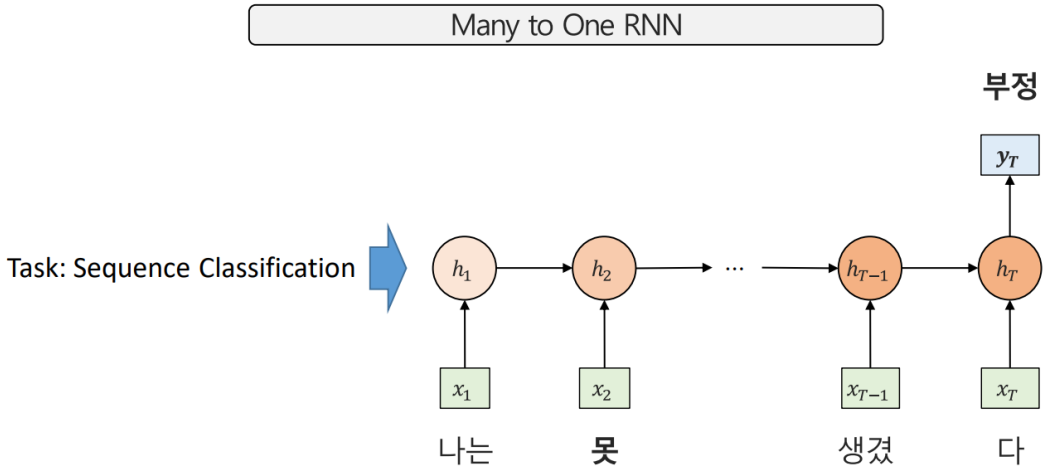
나는	잘	못	생겼	다
1	0	0	0	0

RNN

❖ RNN 구조

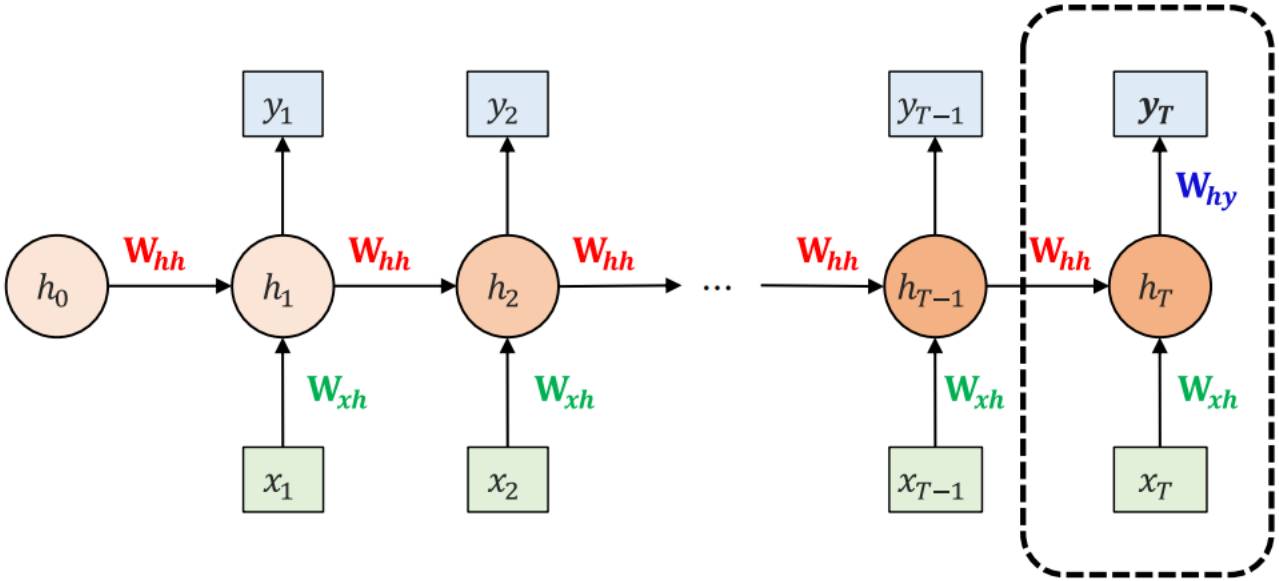


나는	잘	못	생겼	다
1	0	0	0	0



RNN

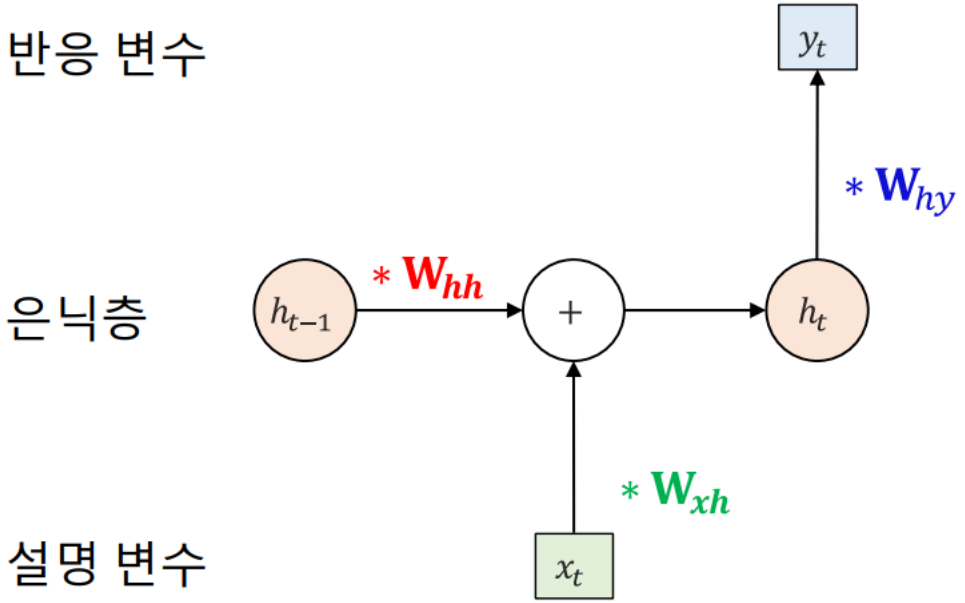
- RNN: t 시점까지 전해온 정보와 t시점 데이터를 가지고 t 시점 Y를 예측
  - t 시점 이전 정보 반영 → W_{hh}
  - t 시점 데이터 반영 → W_{xh}
  - t 시점의 y를 예측 → W_{hy}



학습의 대상(parameters): (W_{xh}, W_{hh}, W_{hy})

RNN

- RNN: t 시점까지 전해온 정보와 t시점 데이터를 가지고 t 시점 Y를 예측
  - t 시점 이전 정보 반영 → W_{hh}
  - t 시점 데이터 반영 → W_{xh}
  - t 시점의 y를 예측 → W_{hy}

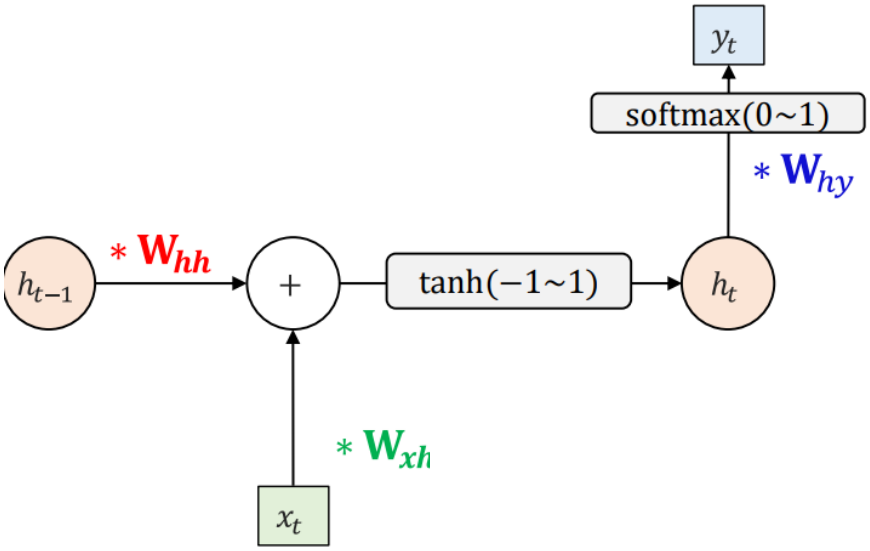


학습의 대상: (W_{xh}, W_{hh}, W_{hy})

RNN

❖ RNN Parameter(학습 대상) : (W_{xh}, W_{hh}, W_{hy})

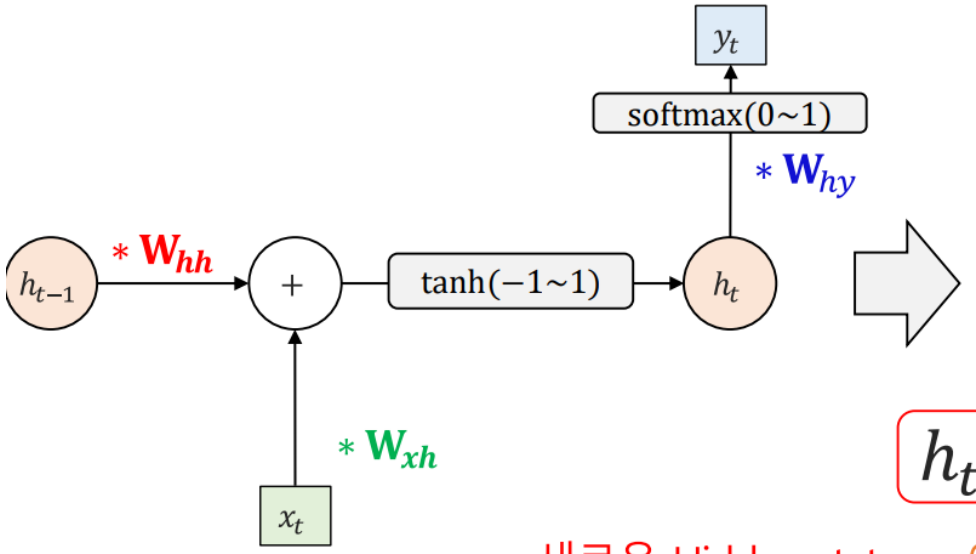
주목할 것: (W_{xh}, W_{hh}, W_{hy})에 t가 없네? → 매 시점 같다!



RNN

❖ RNN Parameter(학습 대상) : (W_{xh}, W_{hh}, W_{hy})

주목할 것: (W_{xh}, W_{hh}, W_{hy})에 t가 없네? → 매 시점 같다!



$$y_t = g(W_{hy}h_t)$$
$$\cdots g(\cdot) = \text{softmax}$$

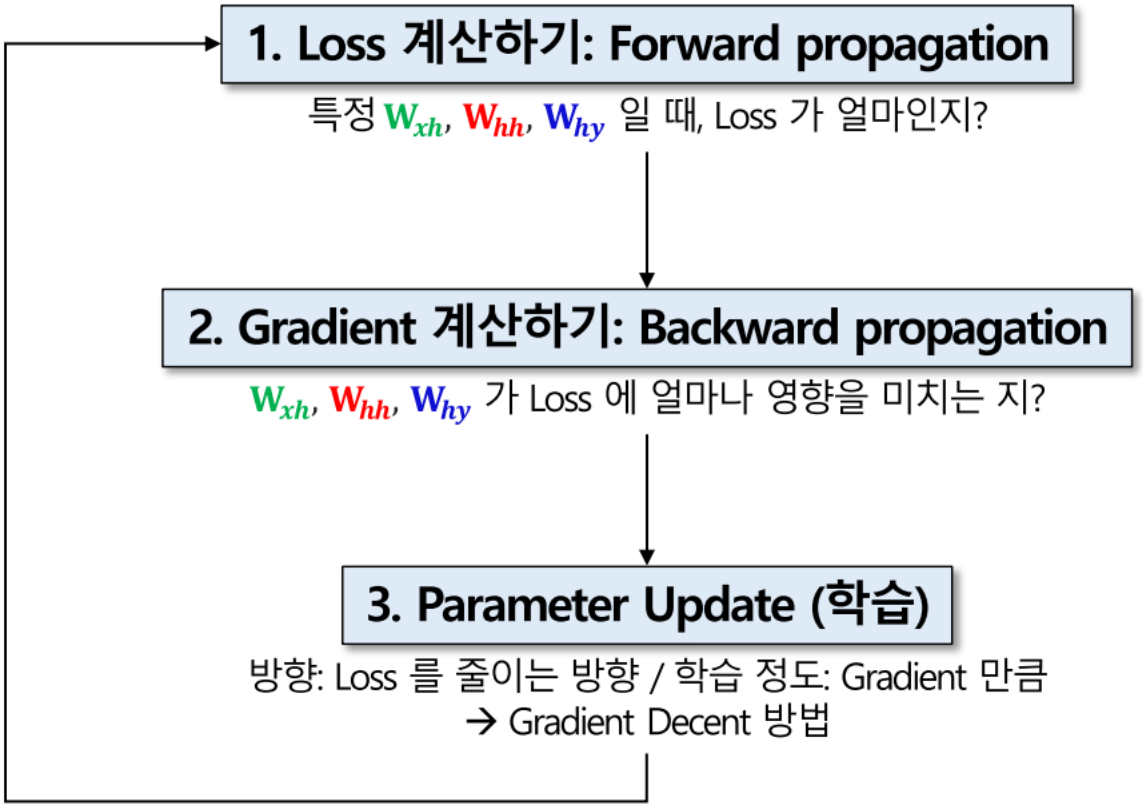
$$h_t = f_W(h_{t-1}, x_t)$$
$$\cdots f(\cdot) = \tanh$$

$$h_t = f(W_{hh}h_{t-1} + W_{xh}x_t)$$

새로운 Hidden state　이전 Hidden state　T시점 input 데이터

RNN

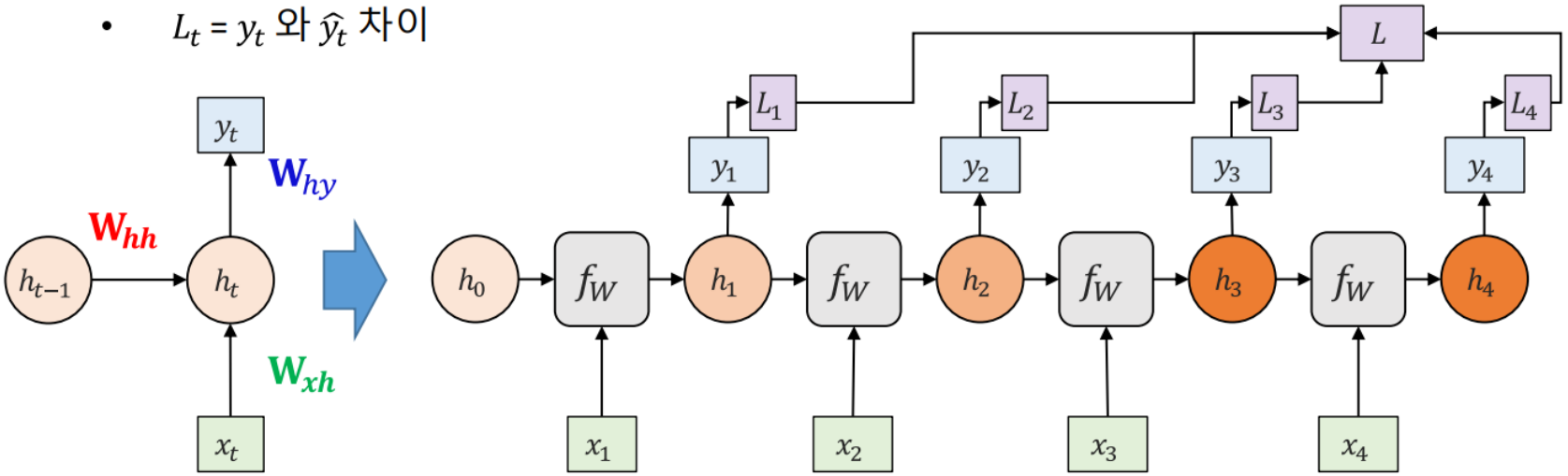
❖ RNN Parameter(학습 대상) : (W_{xh}, W_{hh}, W_{hy})



RNN

❖ RNN Parameter(학습 대상) : (W_{xh}, W_{hh}, W_{hy})

- $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$ → 각 weight 곱하고 더한 값을 -1~1사이로 정규화 한 hidden state 생성
- $y_t = \text{softmax}(W_{hy}h_t)$ → Hidden state에 weight 곱한 값을 0~1사이로 정규화 한 output 생성
- L_t = y_t 와 $\hat{y}_t$ 차이



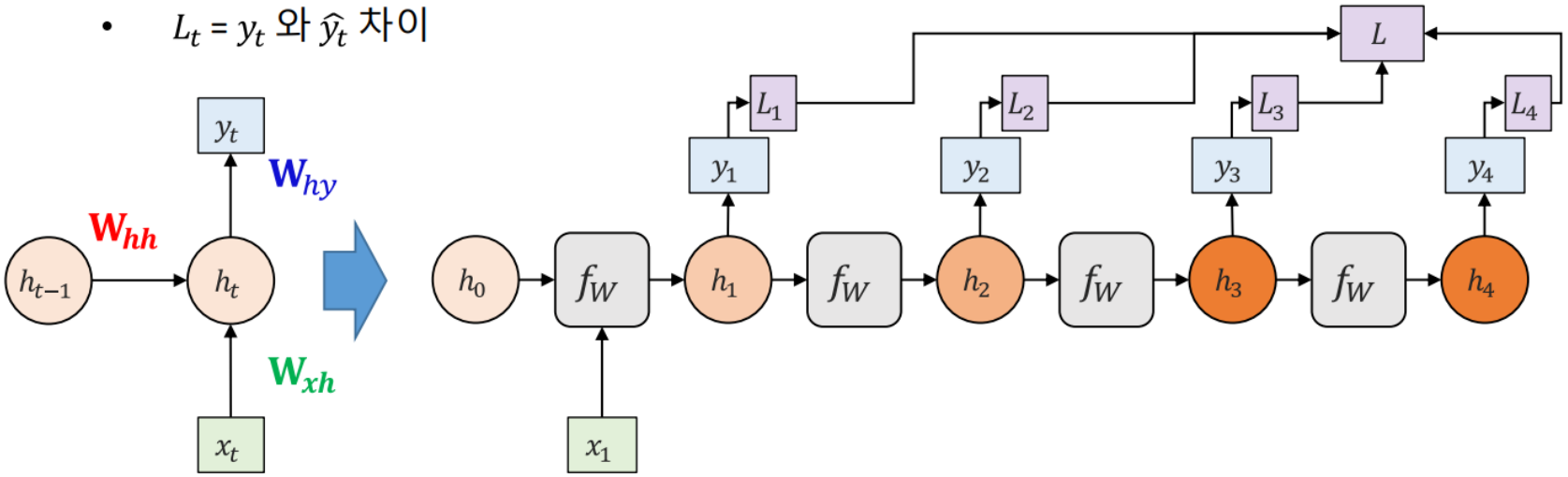
Many to Many RNN

1. Loss 계산하기: Forward propagation

RNN

❖ RNN Parameter(학습 대상) : (W_{xh}, W_{hh}, W_{hy})

- $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$ → 각 weight 곱하고 더한 값을 -1~1사이로 정규화 한 hidden state 생성
- $y_t = \text{softmax}(W_{hy}h_t)$ → Hidden state에 weight 곱한 값을 0~1사이로 정규화 한 output 생성
- L_t = y_t 와 $\hat{y}_t$ 차이



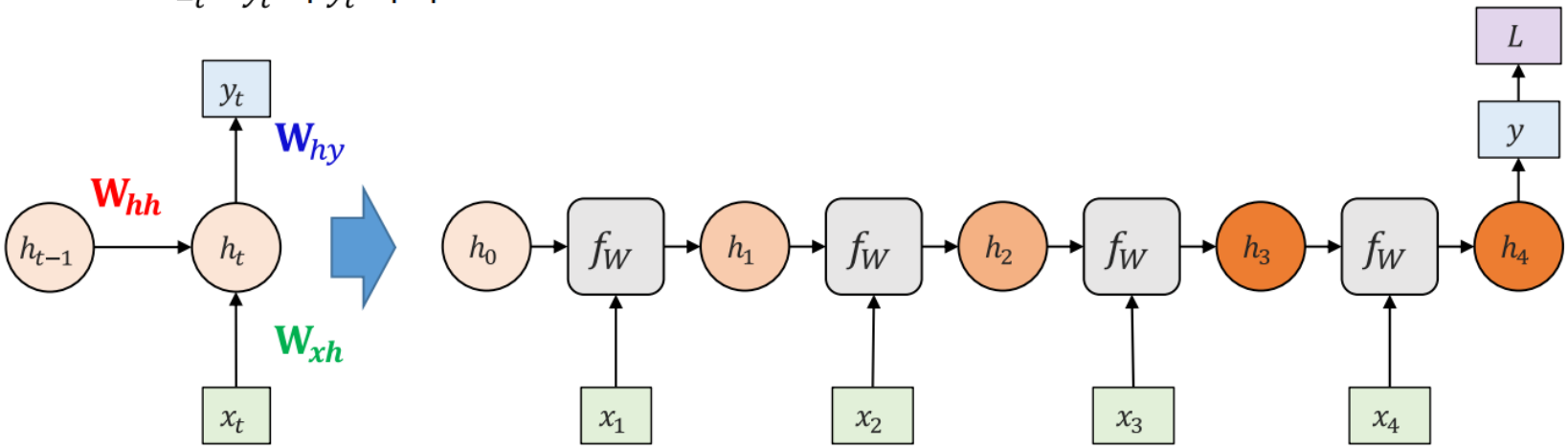
One to Many RNN

1. Loss 계산하기: Forward propagation

RNN

❖ RNN Parameter(학습 대상) : (W_{xh}, W_{hh}, W_{hy})

- $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$ → 각 weight 곱하고 더한 값을 -1~1사이로 정규화 한 hidden state 생성
- $y_t = \text{softmax}(W_{hy}h_t)$ → Hidden state에 weight 곱한 값을 0~1사이로 정규화 한 output 생성
- L_t = y_t 와 $\hat{y}_t$ 차이



Many to One RNN

1. Loss 계산하기: Forward propagation

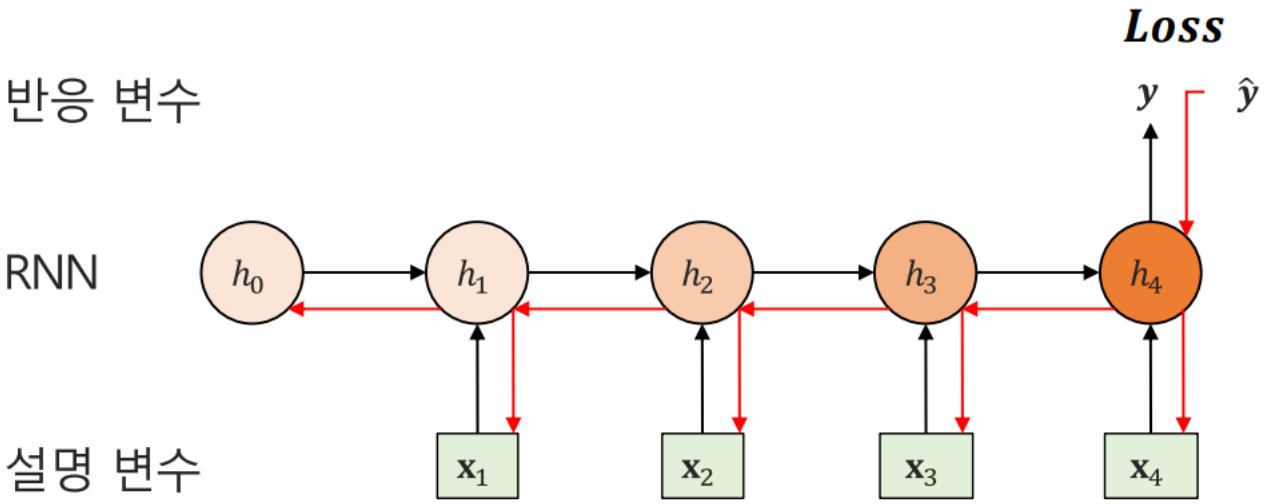
RNN

2. Gradient 계산하기: Backward propagation

❖ Backward Propagation Trough Time (BPTT) in RNN

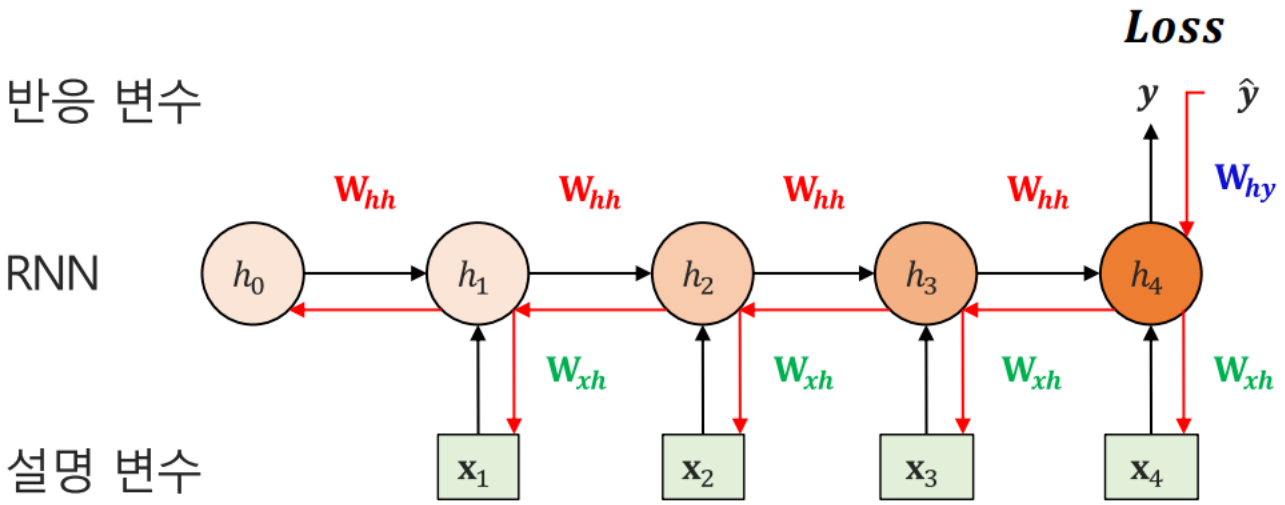
- 학습 대상 (W_{xh}, W_{hh}, W_{hy}, parameters)

시간 흐름에 따라 파라미터 업데이트



RNN

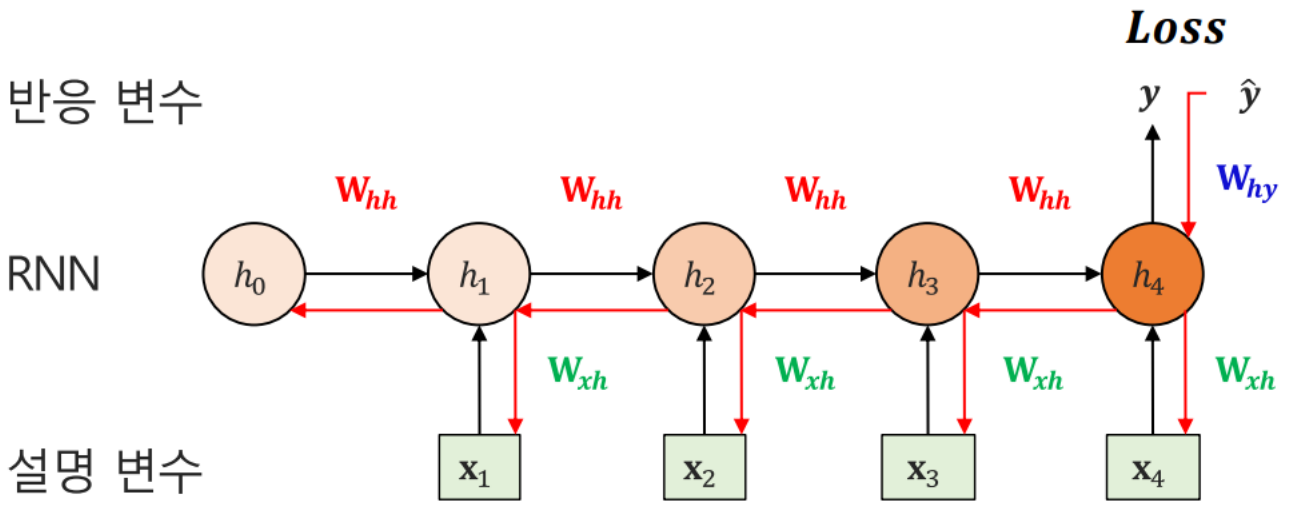
- Backward Propagation Trough Time (BPTT) in RNN
- RNN Parameter(학습 대상) : (W_{xh}, W_{hh}, W_{hy})
  - $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$ → 각 weight 곱하고 더한 값을 -1~1사이로 정규화 한 hidden state 생성
  - $y_t = \text{softmax}(W_{hy}h_t)$ → Hidden state에 weight 곱한 값을 0~1사이로 정규화 한 output 생성
  - L_t = y_t 와 $\hat{y}_t$ 차이



RNN

- Backward Propagation Trough Time (BPTT) in RNN
- RNN Parameter(학습 대상) : (W_{xh}, W_{hh}, W_{hy})
  - $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$ → 각 weight 곱하고 더한 값을 -1~1사이로 정규화 한 hidden state 생성
  - $y_t = \text{softmax}(W_{hy}h_t)$ → Hidden state에 weight 곱한 값을 0~1사이로 정규화 한 output 생성
  - L_t = y_t 와 $\hat{y}_t$ 차이

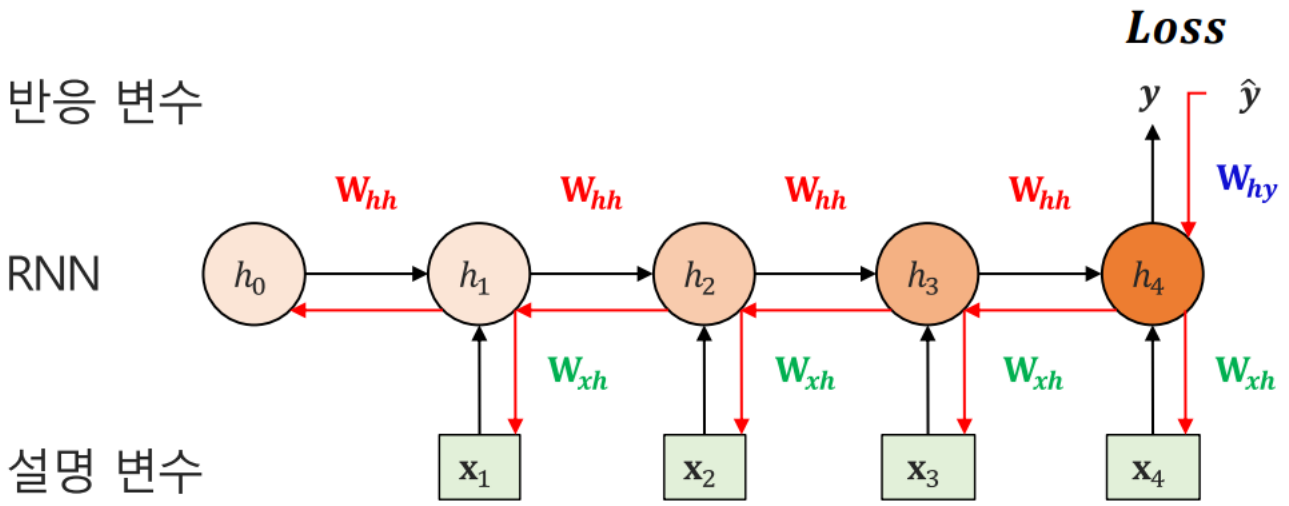
$$\frac{\partial Loss}{\partial \mathbf{W}_{hy}} = \frac{\partial L_t}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial \mathbf{W}_{hy} h_t} \times \frac{\partial \mathbf{W}_{hy} h_t}{\partial \mathbf{W}_{hy}}$$



RNN

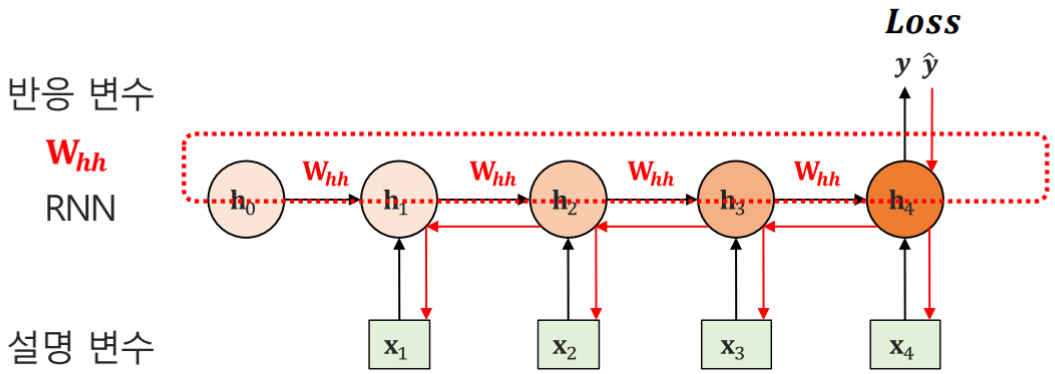
- Backward Propagation Trough Time (BPTT) in RNN
- RNN Parameter(학습 대상) : (W_{xh}, W_{hh}, W_{hy})
  - $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$ → 각 weight 곱하고 더한 값을 -1~1사이로 정규화 한 hidden state 생성
  - $y_t = \text{softmax}(W_{hy}h_t)$ → Hidden state에 weight 곱한 값을 0~1사이로 정규화 한 output 생성
  - L_t = y_t 와 $\hat{y}_t$ 차이

$$\frac{\partial Loss}{\partial \mathbf{W}_{hy}} = \frac{\partial L_t}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial \mathbf{W}_{hy} h_t} \times \frac{\partial \mathbf{W}_{hy} h_t}{\partial \mathbf{W}_{hy}}$$



RNN

- Backward Propagation Trough Time (BPTT) in RNN
- RNN Parameter(학습 대상) : (W_{xh}, W_{hh}, W_{hy})
  - $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$ → 각 weight 곱하고 더한 값을 -1~1사이로 정규화 한 hidden state 생성
  - $y_t = \text{softmax}(W_{hy}h_t)$ → Hidden state에 weight 곱한 값을 0~1사이로 정규화 한 output 생성
  - L_t = y_t 와 $\hat{y}_t$ 차이



시점 4 으로부터 전해진 영향 고려

$$\frac{\partial Loss}{\partial \mathbf{W_{hh}}} = \frac{\partial Loss}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial \mathbf{h}_4} \times \frac{\partial \mathbf{h}_4}{\partial \mathbf{W_{hh}}} +$$

시점 3 으로부터 전해진 영향 고려

$$\frac{\partial Loss}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial \mathbf{h}_4} \times \frac{\partial \mathbf{h}_4}{\partial \mathbf{h}_3} \times \frac{\partial \mathbf{h}_3}{\partial \mathbf{W_{hh}}} +$$

시점 2 으로부터 전해진 영향 고려

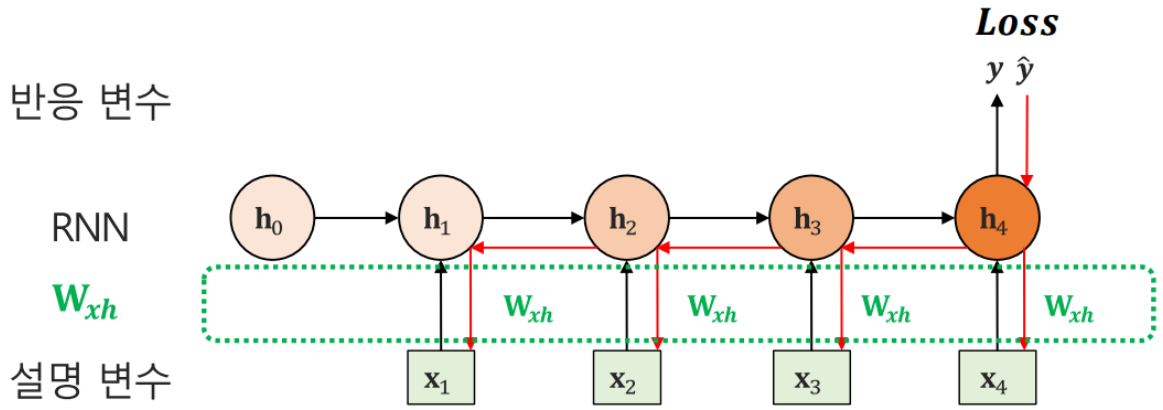
$$\frac{\partial Loss}{\partial \hat{y}_t} \times \frac{\partial y}{\partial \mathbf{h}_4} \times \frac{\partial \mathbf{h}_4}{\partial \mathbf{h}_3} \times \frac{\partial \mathbf{h}_3}{\partial \mathbf{h}_2} \times \frac{\partial \mathbf{h}_2}{\partial \mathbf{W_{hh}}} +$$

시점 1 으로부터 전해진 영향 고려

$$\frac{\partial Loss}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial \mathbf{h}_4} \times \frac{\partial \mathbf{h}_4}{\partial \mathbf{h}_3} \times \frac{\partial \mathbf{h}_3}{\partial \mathbf{h}_2} \times \frac{\partial \mathbf{h}_2}{\partial \mathbf{h}_1} \times \frac{\partial \mathbf{h}_2}{\partial \mathbf{h}_1}$$

RNN

- Backward Propagation Trough Time (BPTT) in RNN
- RNN Parameter(학습 대상) : (W_{xh}, W_{hh}, W_{hy})
  - $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$ → 각 weight 곱하고 더한 값을 -1~1사이로 정규화 한 hidden state 생성
  - $y_t = \text{softmax}(W_{hy}h_t)$ → Hidden state에 weight 곱한 값을 0~1사이로 정규화 한 output 생성
  - L_t = y_t 와 $\hat{y}_t$ 차이



시점 4 으로부터 전해진 영향 고려

$$\frac{\partial Loss}{\partial W_{xh}} = \frac{\partial Loss}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial h_4} \times \frac{\partial h_4}{\partial W_{xh}} +$$

시점 3 으로부터 전해진 영향 고려

$$\frac{\partial Loss}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial h_4} \times \frac{\partial h_4}{\partial h_3} \times \frac{\partial h_3}{\partial W_{xh}} +$$

시점 2 으로부터 전해진 영향 고려

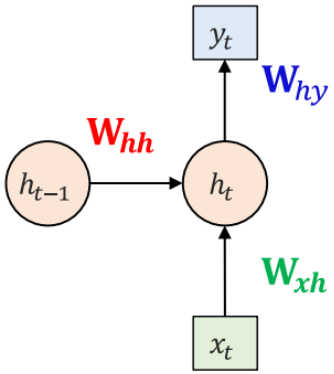
$$\frac{\partial Loss}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial h_4} \times \frac{\partial h_4}{\partial h_3} \times \frac{\partial h_3}{\partial h_2} \times \frac{\partial h_2}{\partial W_{xh}} +$$

시점 1 으로부터 전해진 영향 고려

$$\frac{\partial Loss}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial h_4} \times \frac{\partial h_4}{\partial h_3} \times \frac{\partial h_3}{\partial h_2} \times \frac{\partial h_2}{\partial h_1} \times \frac{\partial h_2}{\partial h_1}$$

RNN

- Backward Propagation Trough Time (BPTT) in RNN
- RNN Parameter(학습 대상) : (W_{xh}, W_{hh}, W_{hy})
  - $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$ → 각 weight 곱하고 더한 값을 -1~1사이로 정규화 한 hidden state 생성
  - $y_t = \text{softmax}(W_{hy}h_t)$ → Hidden state에 weight 곱한 값을 0~1사이로 정규화 한 output 생성
  - L_t = y_t 와 $\hat{y}_t$ 차이



$$\frac{\partial Loss}{\partial \mathbf{W_{hy}}} = \frac{\partial L_t}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial \mathbf{W_{hy}} h_t} \times \frac{\partial \mathbf{W_{hy}} V h_t}{\partial \mathbf{W_{hy}}}$$

$$\frac{\partial Loss}{\partial \mathbf{W_{hh}}} = \sum_{k=0}^{t} \left(\frac{\partial L}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial h_t} \times \frac{\partial h_t}{\partial h_k} \times \frac{\partial h_k}{\partial \mathbf{W_{hh}}} \right)$$

$$\frac{\partial Loss}{\partial \mathbf{W}_{xh}} = \sum_{k=0}^{t} \left(\frac{\partial L}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial h_t} \times \frac{\partial h_t}{\partial h_k} \times \frac{\partial h_k}{\partial \mathbf{W}_{xh}} \right)$$

회귀 문제: Loss function$(MSE) = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$

분류 문제: Loss function$(cross\ entropy) = \sum_{c=1}^{M} y_{o,c} log(p_{o,c})$

RNN

❖ Parameter update in RNN

- 학습 대상 (W_{xh}, W_{hh}, W_{hy}, parameters) → BPTT를 통해 Gradient(기여도) 계산
- 학습률 (η, learning rate): 기여도를 얼마나 반영할 것인지 결정(예: η=0.01)

① A: W_{hy}의 gradient(기여도) = $\frac{\partial L_t}{\partial W_{hy}}$

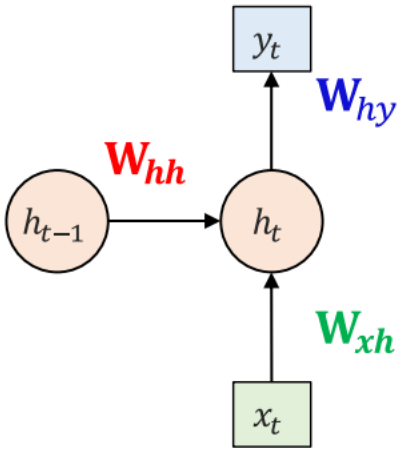
→ $W_{hy}^{new} = W_{hy}^{old} - \eta * A$

② B: W_{hh}의 gradient(기여도) = $\frac{\partial L_t}{\partial W_{hh}}$

→ $W_{hh}^{new} = W_{hh}^{old} - \eta * B$

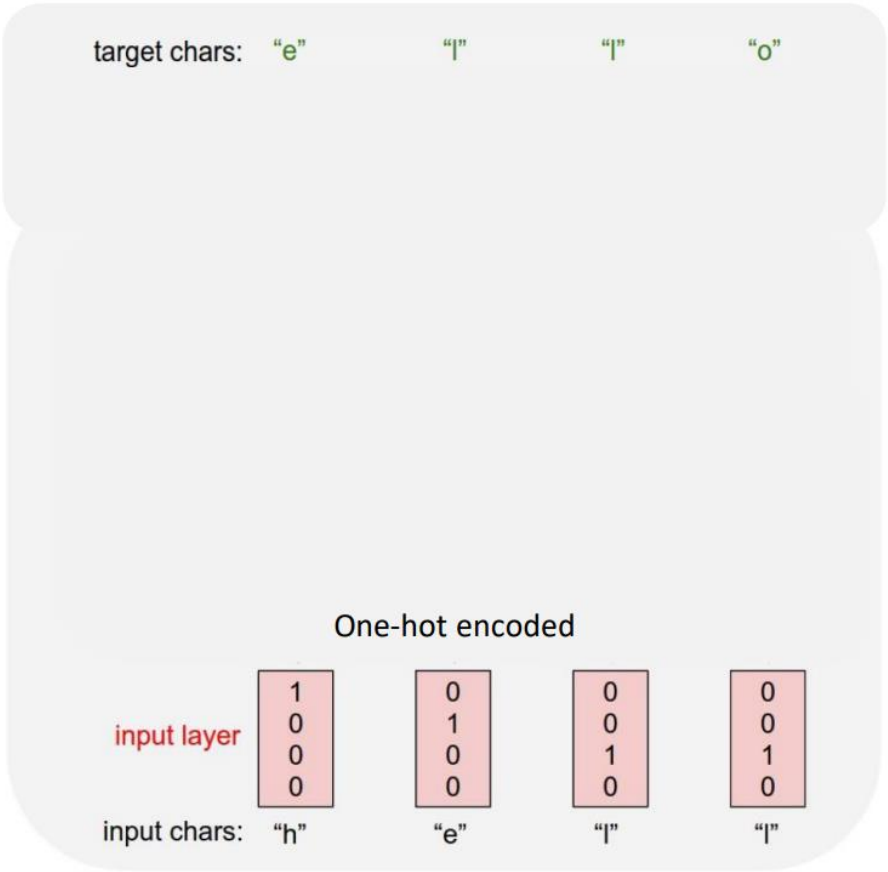
③ C : W_{xh}의 gradient(기여도) = $\frac{\partial L_t}{\partial W_{xh}}$

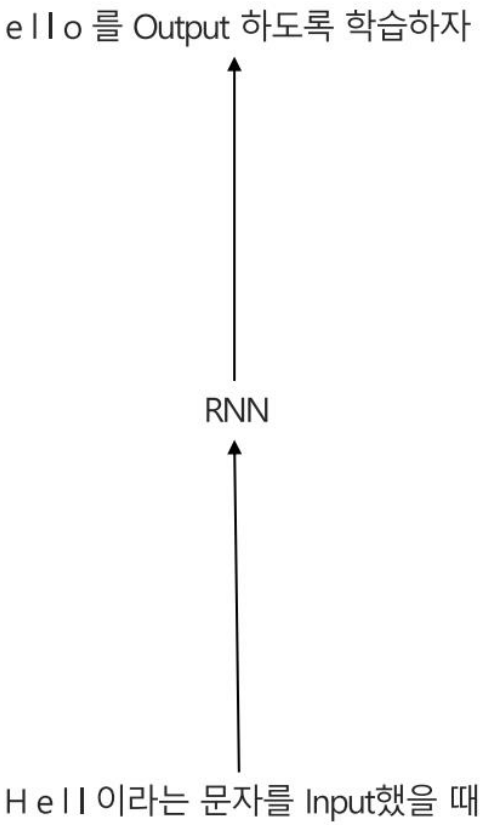
→ $W_{xh}^{new} = W_{xh}^{old} - \eta * C$



RNN

❖ RNN 학습 예제: Many to Many RNN for Language modeling

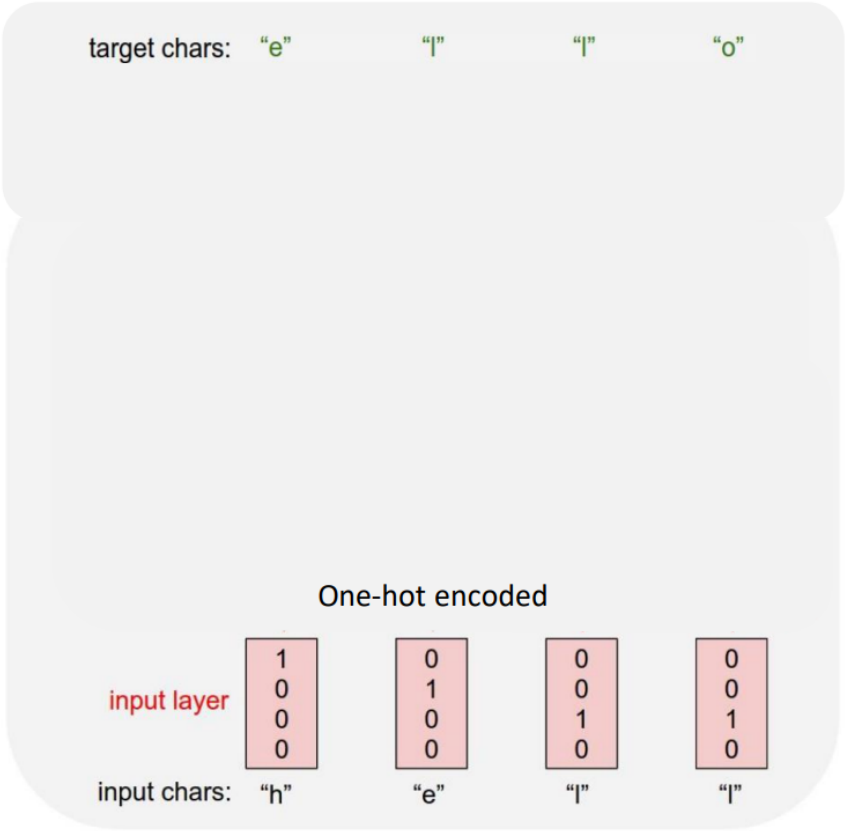




- https://cs224d.stanford.edu/

RNN

❖ RNN 학습 예제: Many to Many RNN for Language modeling



1. Loss 계산하기: Forward propagation

Hidden size=4 # output from RNN

Batch_size = 1 # one sentence

Input_dim = 4 # one-hot size

Sequence length = 4 # len(hell)== 4

$$y_t = W_{hy} h_t$$

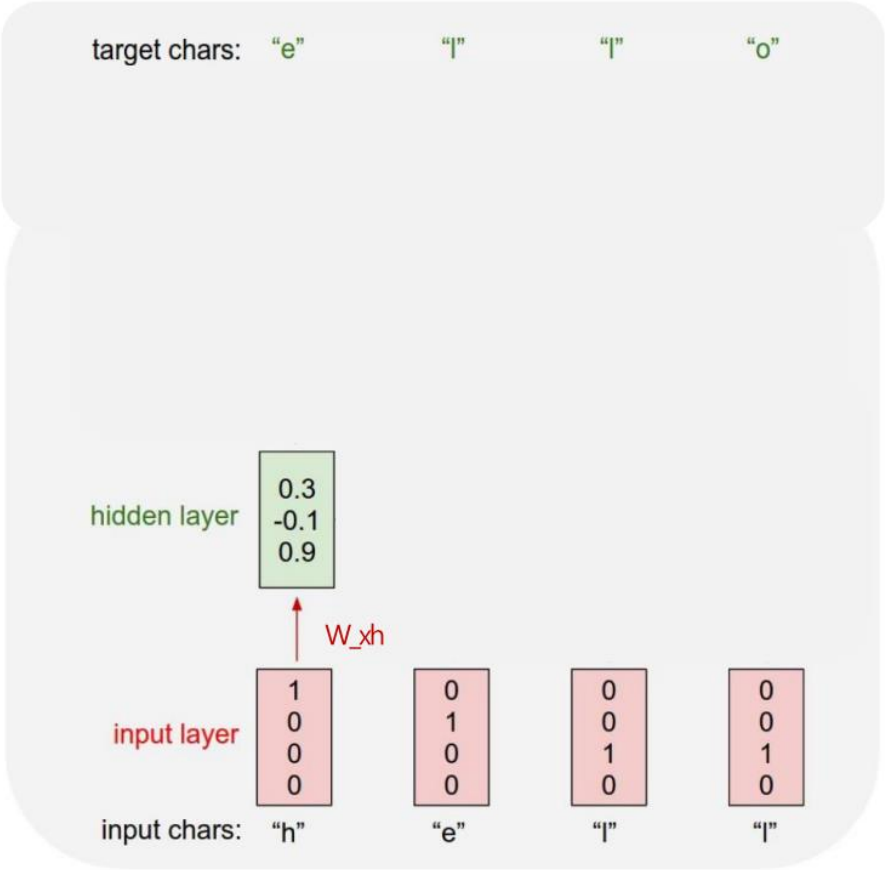
$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t)$$

$$x_1 = [1 \quad 0 \quad 0 \quad 0]$$

- https://cs224d.stanford.edu/

RNN

❖ RNN 학습 예제: Many to Many RNN for Language modeling



1. Loss 계산하기: Forward propagation

Hidden size=4 # output from RNN

Batch_size = 1 # one sentence

Input_dim = 4 # one-hot size

Sequence length = 4 # len(hell)== 4

$$y_t = W_{hy} h_t$$

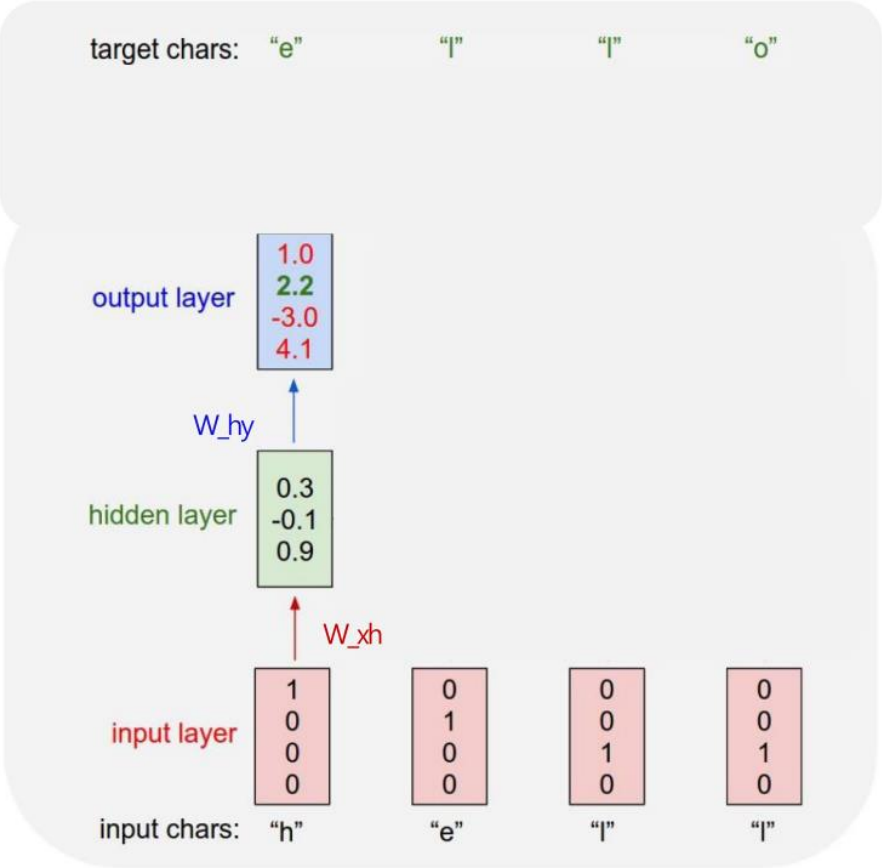
$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t)$$

$$x_1 = [1 \quad 0 \quad 0 \quad 0]$$

- https://cs224d.stanford.edu/

RNN

❖ RNN 학습 예제: Many to Many RNN for Language modeling



- https://cs224d.stanford.edu/

1. Loss 계산하기: Forward propagation

Hidden size=4 # output from RNN

Batch_size = 1 # one sentence

Input_dim = 4 # one-hot size

Sequence length = 4 # len(hell)== 4

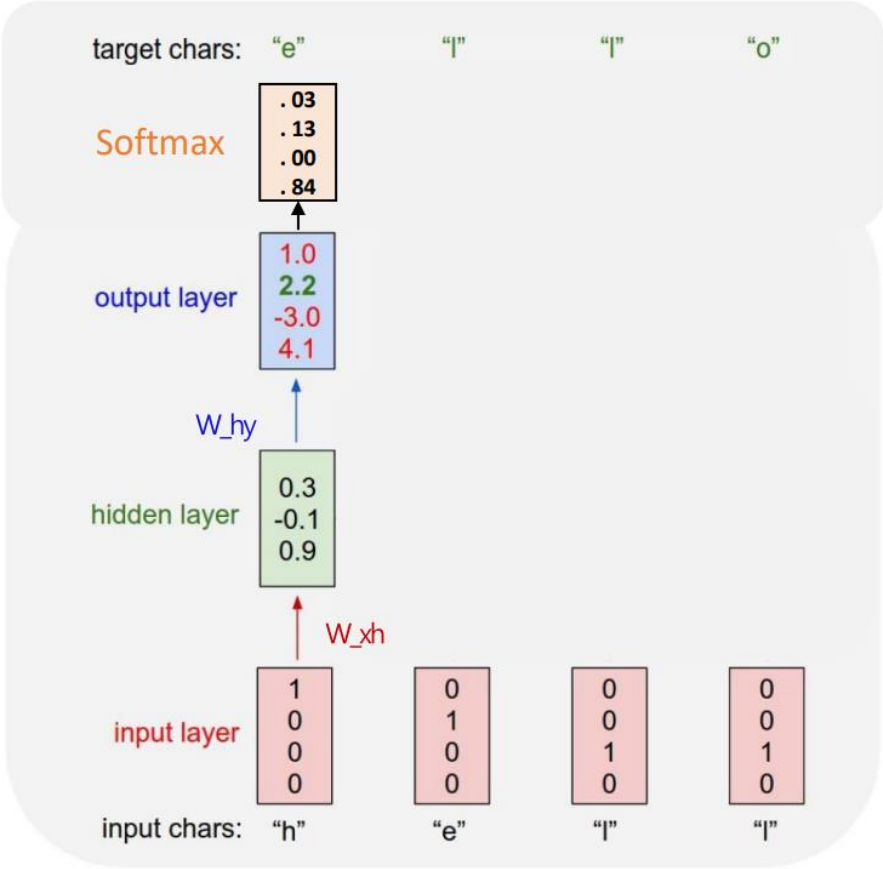
$$y_t = \boldsymbol{W_{hy}} h_t$$

$$h_t = \tanh(\boldsymbol{W_{hh}} h_{t-1} + \boldsymbol{W_{xh}} x_t)$$

$$x_1 = [1 \quad 0 \quad 0 \quad 0]$$

RNN

❖ RNN 학습 예제: Many to Many RNN for Language modeling



- https://cs224d.stanford.edu/

1. Loss 계산하기: Forward propagation

Hidden size=4 # output from RNN

Batch_size = 1 # one sentence

Input_dim = 4 # one-hot size

Sequence length = 4 # len(hell)== 4

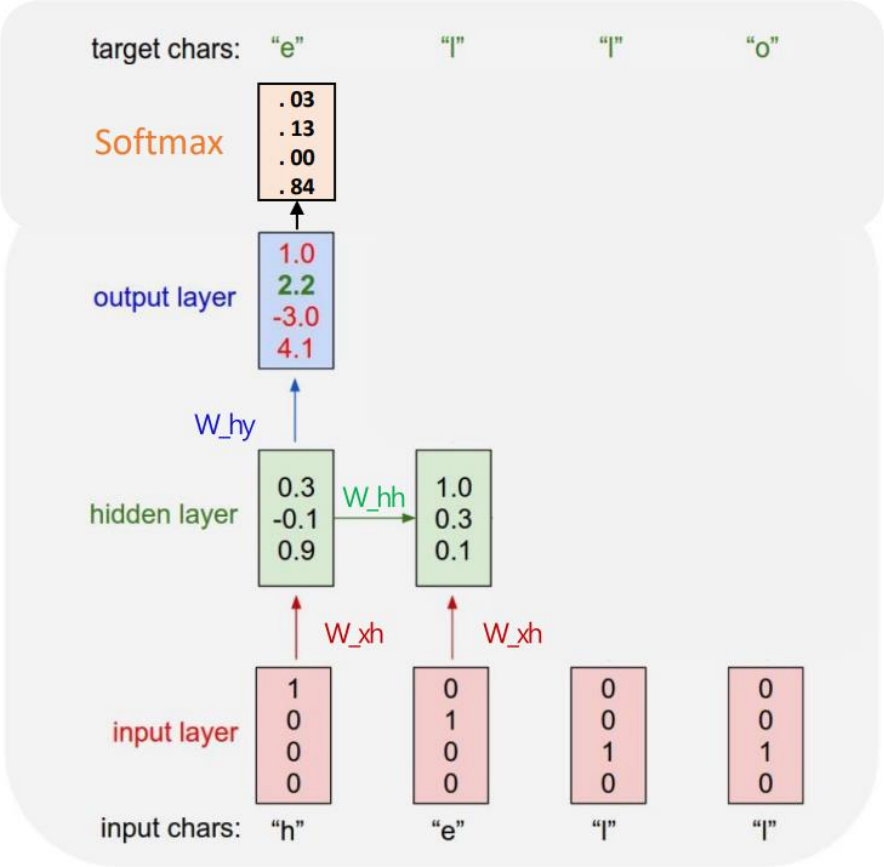
$$y_t = \boldsymbol{W_{hy}} h_t$$

$$h_t = \tanh(\boldsymbol{W_{hh}} h_{t-1} + \boldsymbol{W_{xh}} x_t)$$

$$x_1 = [1 \quad 0 \quad 0 \quad 0]$$

RNN

❖ RNN 학습 예제: Many to Many RNN for Language modeling



- https://cs224d.stanford.edu/

1. Loss 계산하기: Forward propagation

Hidden size=4 # output from RNN

Batch_size = 1 # one sentence

Input_dim = 4 # one-hot size

Sequence length = 4 # len(hell)== 4

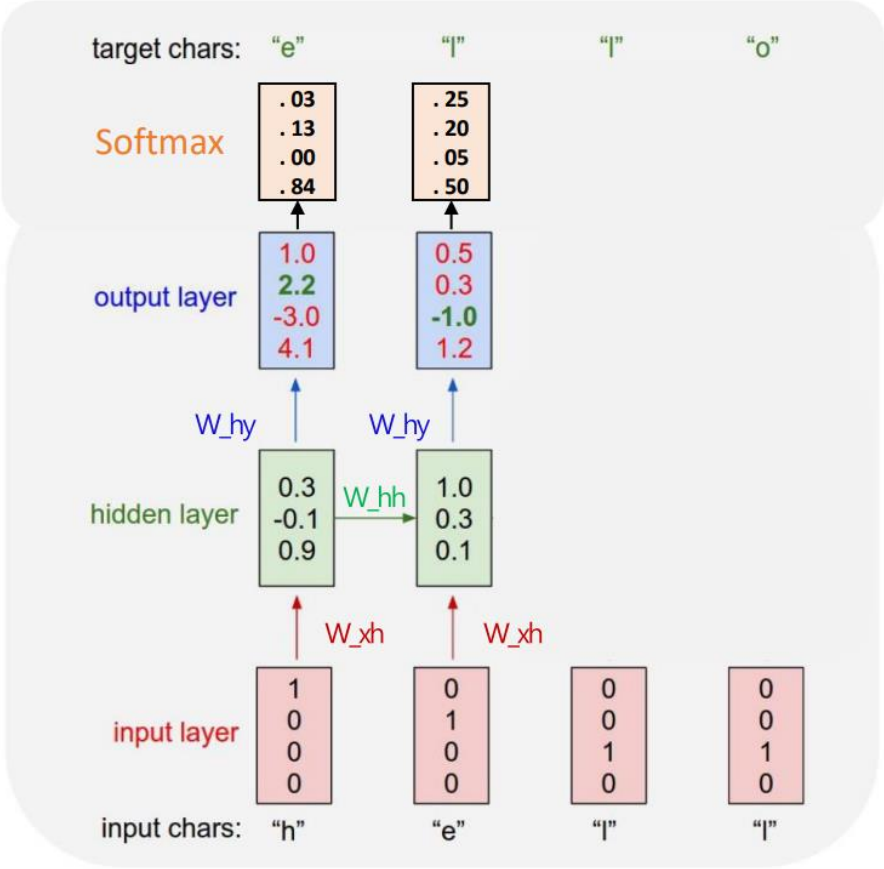
$$y_t = \boldsymbol{W_{hy}} h_t$$

$$h_t = \tanh(\boldsymbol{W_{hh}} h_{t-1} + \boldsymbol{W_{xh}} x_t)$$

$$x_2 = [0 \quad 1 \quad 0 \quad 0]$$

RNN

❖ RNN 학습 예제: Many to Many RNN for Language modeling



1. Loss 계산하기: Forward propagation

Hidden size=4 # output from RNN

Batch_size = 1 # one sentence

Input_dim = 4 # one-hot size

Sequence length = 4 # len(hell)== 4

$$y_t = \boldsymbol{W_{hy}} h_t$$

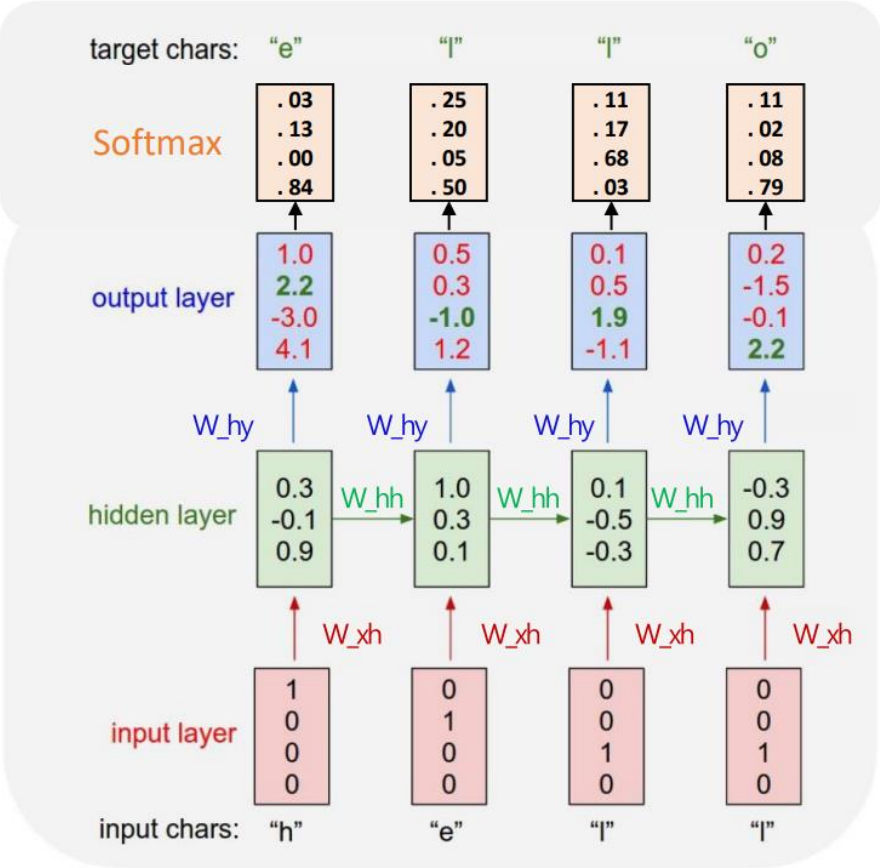
$$h_t = \tanh(\boldsymbol{W_{hh}} h_{t-1} + \boldsymbol{W_{xh}} x_t)$$

$$x_2 = [0 \quad 1 \quad 0 \quad 0]$$

- https://cs224d.stanford.edu/

RNN

❖ RNN 학습 예제: Many to Many RNN for Language modeling



- https://cs224d.stanford.edu/

1. Loss 계산하기: Forward propagation

Hidden size=4 # output from RNN

Batch_size = 1 # one sentence

Input_dim = 4 # one-hot size

Sequence length = 4 # len(hell)== 4

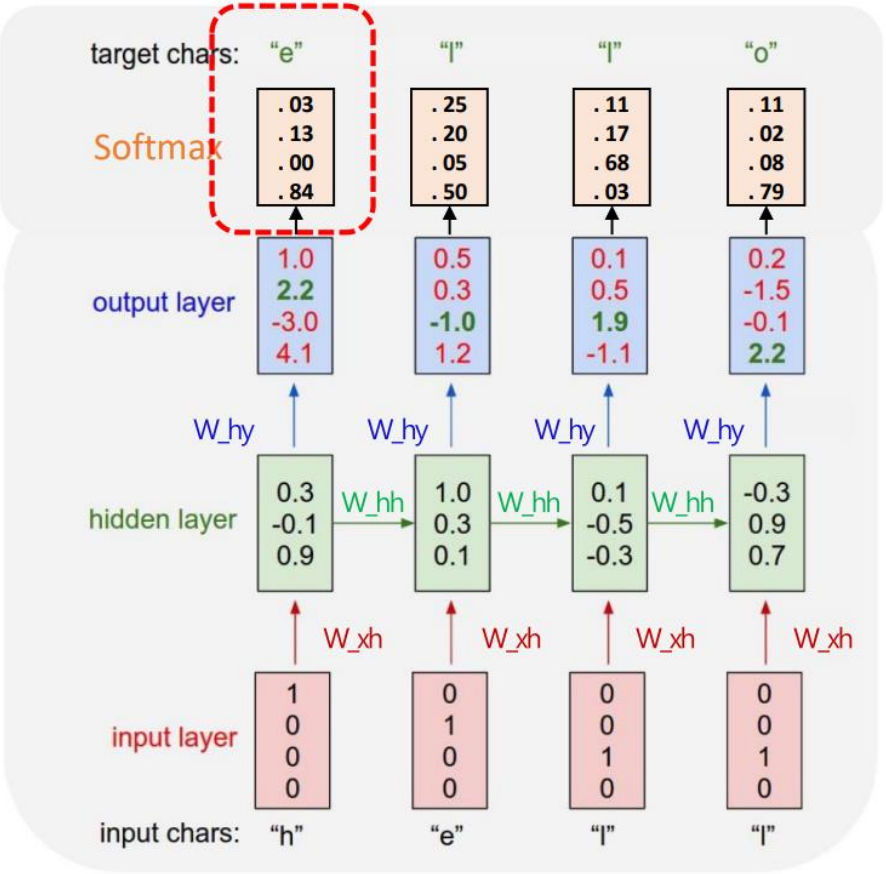
$$y_t = \boldsymbol{W_{hy}} h_t$$

$$h_t = \tanh(\boldsymbol{W_{hh}} h_{t-1} + \boldsymbol{W_{xh}} x_t)$$

$$x_4 = [0 \quad 0 \quad 0 \quad 1]$$

RNN

❖ RNN 학습 예제: Many to Many RNN for Language modeling



1. Loss 계산하기: Forward propagation

h
실제← e
l
예측 ← o

.03
.13
.00
.84

Error!

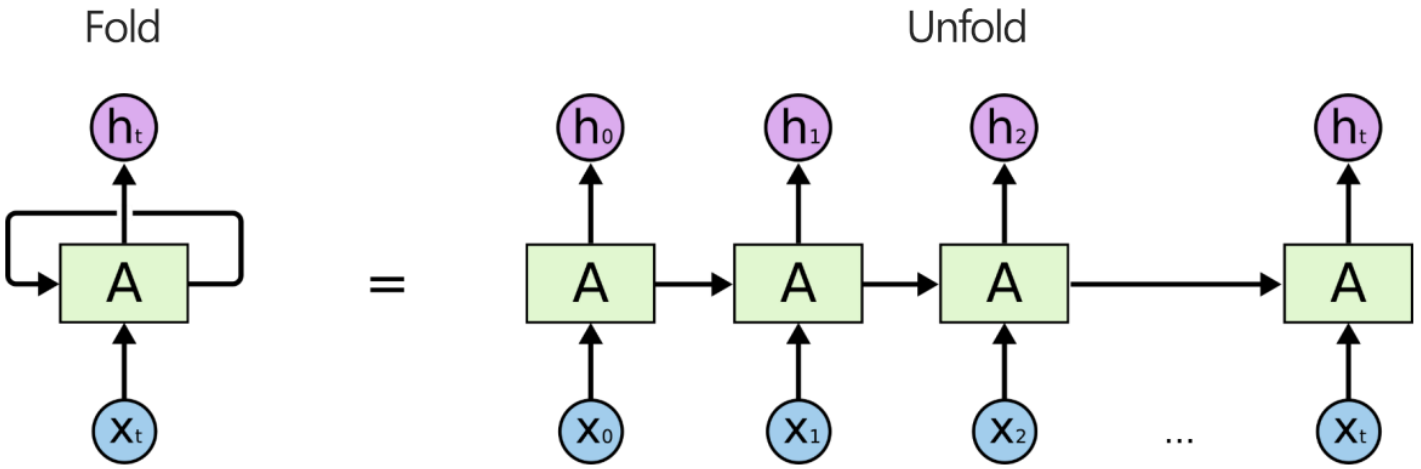
2. Gradient 계산하기: Backward propagation

- https://cs224d.stanford.edu/

RNN 한계

❖ RNN은 긴 시퀀스를 모델링하기 어려움

- Vanishing gradient



$$h_1 = \tanh(Ux_1 + Wh_0)$$

$$h_2 = \tanh(Ux_2 + Wh_1) \quad \cdots$$

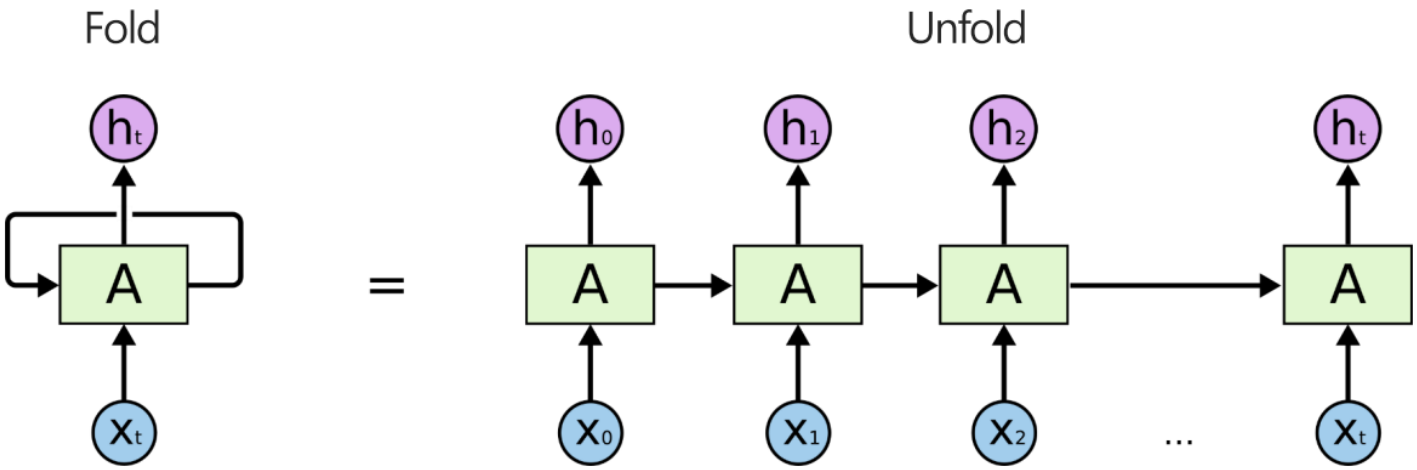
$$h_t = \tanh(Ux_t + Wh_{t-1})$$

https://colah.github.io/posts/2015-08-Understanding-LSTMs/

RNN 한계

❖ RNN은 긴 시퀀스를 모델링하기 어려움

- Vanishing gradient



$$h_t = \tanh(Ux_t + Wh_{t-1})$$

"tanh 너무 많아!!"

$$= \tanh\Big(Ux_t + W\big(\tanh\big(Ux_{t-1} + W\big(\tanh(Ux_{t-2} + W(\tanh(\dots)))\ \dots\big)\big)\Big)\Big)$$

https://colah.github.io/posts/2015-08-Understanding-LSTMs/

RNN

❖ 길이기 긴 sequence 의 경우 Gradient Vanishing / Exploding 문제 발생

- Vanishing gradient: 다른 모델 사용 →(LSTM, GRU 등)
- Exploding gradient: Clip gradient(기여도에 한계선 부여)

Long-Term Dependency problem

h_t가 의존하는 h_{t-1}이 의존하는 h_{t-2}가 의존하는 …

시점 4 으로부터 전해진 영향 고려

$$\frac{\partial Loss}{\partial \mathbf{W_{hh}}} = \frac{\partial Loss}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial \mathbf{h}_4} \times \frac{\partial \mathbf{h}_4}{\partial \mathbf{W_{hh}}} +$$

시점 3 으로부터 전해진 영향 고려

$$\frac{\partial Loss}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial \mathbf{h}_4} \times \frac{\partial \mathbf{h}_4}{\partial \mathbf{h}_3} \times \frac{\partial \mathbf{h}_3}{\partial \mathbf{W_{hh}}} +$$

시점 2 으로부터 전해진 영향 고려

$$\frac{\partial Loss}{\partial \hat{y}_t} \times \frac{\partial y}{\partial \mathbf{h}_4} \times \frac{\partial \mathbf{h}_4}{\partial \mathbf{h}_3} \times \frac{\partial \mathbf{h}_3}{\partial \mathbf{h}_2} \times \frac{\partial \mathbf{h}_2}{\partial \mathbf{W_{hh}}} +$$

시점 1 으로부터 전해진 영향 고려

$$\frac{\partial Loss}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial \mathbf{h}_4} \times \frac{\partial \mathbf{h}_4}{\partial \mathbf{h}_3} \times \frac{\partial \mathbf{h}_3}{\partial \mathbf{h}_2} \times \frac{\partial \mathbf{h}_2}{\partial \mathbf{h}_1} \times \frac{\partial \mathbf{h}_2}{\partial \mathbf{h}_1}$$

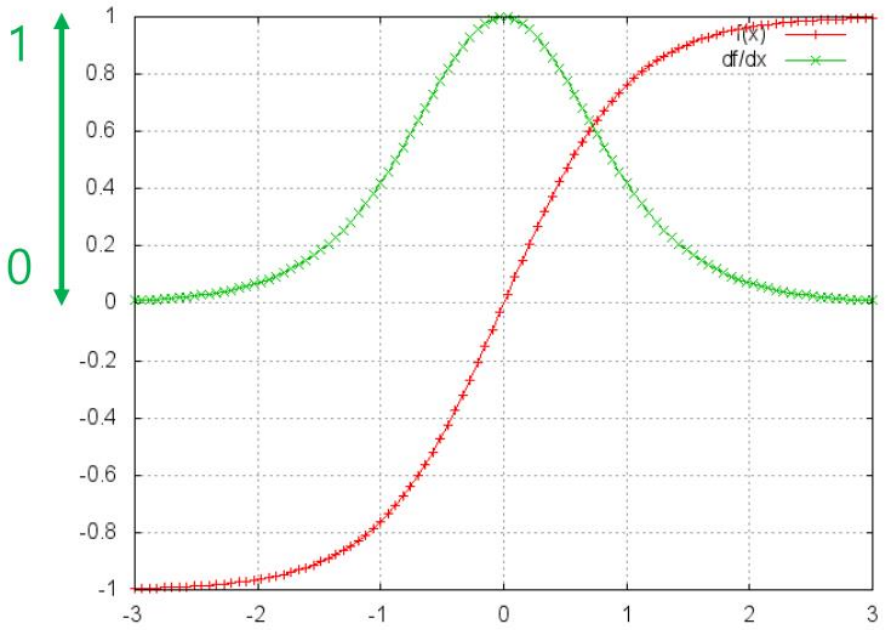
RNN 한계

❖ RNN은 긴 시퀀스를 모델링하기 어려움

- Vanishing gradient: 긴 시퀀스의 경우, gradient가 0으로 수렴하는 문제 발생

  = Back-propagation 수행 안됨

  = 모델 학습 안됨!



tanh(x)

$\frac{d}{dx}\tanh(x)$

RNN 한계

❖ RNN은 긴 시퀀스를 모델링하기 어려움

- Vanishing gradient: 긴 시퀀스의 경우, gradient가 0으로 수렴하는 문제 발생
  = 장기 의존성 문제(Long term dependency)

안녕하세요, 저는 데이터 마이닝 연구실 이지윤입니다.

제가 제일 좋아하는 음식은 계란말이랑 만두고,

... T M I ...

데이터 마이닝

앞으로도 ------- 과 관련된 공부를 열심히 할 계획입니다.

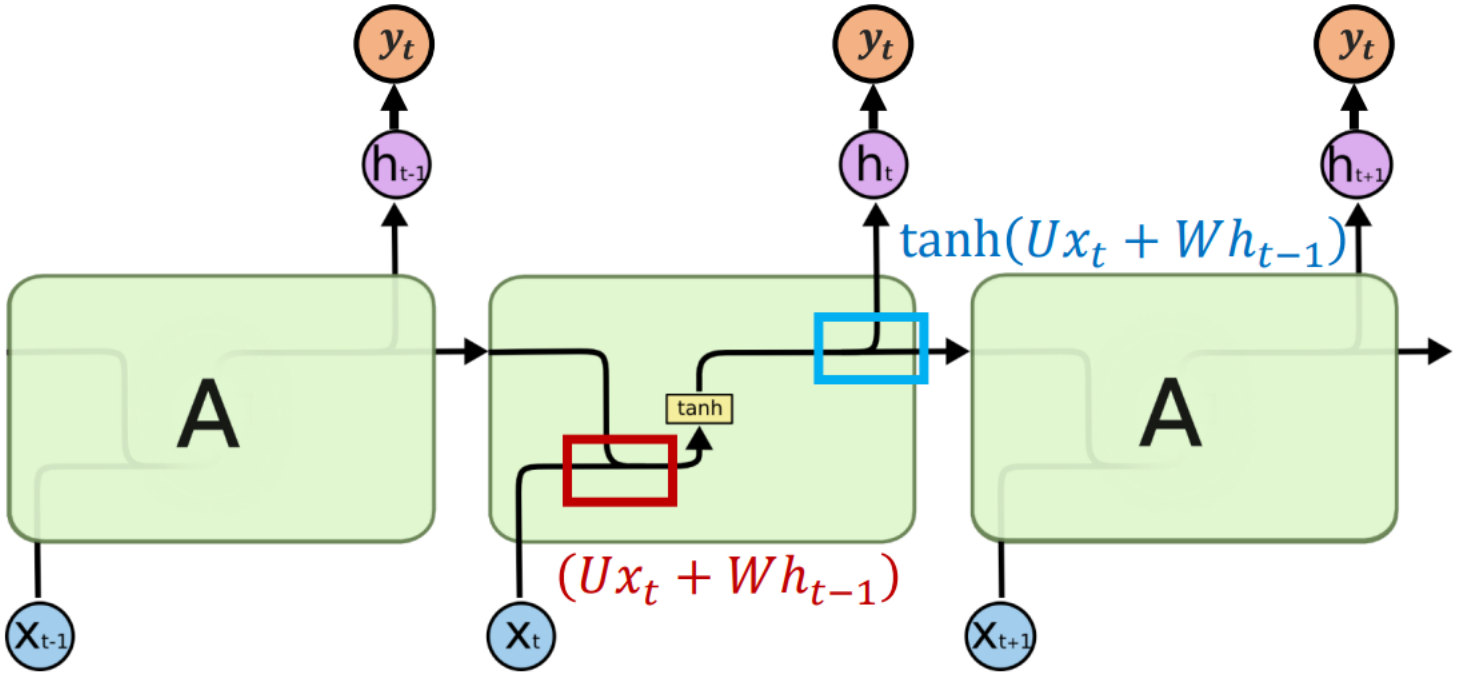
RNN 한계

- RNN은 긴 시퀀스를 모델링하기 어려움
  - Vanishing gradient: 긴 시퀀스의 경우, gradient가 0으로 수렴하는 문제 발생
  - 관련 정보와 그 정보를 사용하는 시점 사이 거리가 길어질수록 RNN 학습능력 저하

https://colah.github.io/posts/2015-08-Understanding-LSTMs/

LSTM (Long-Shor Term Memory)

❖ Standard RNN

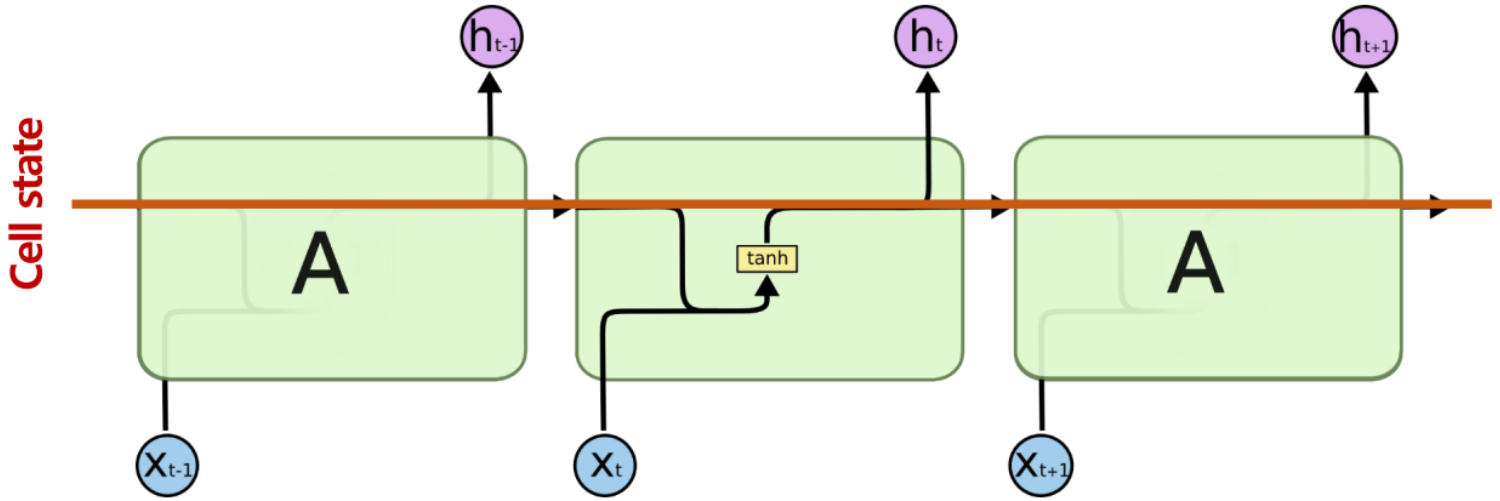


https://colah.github.io/posts/2015-08-Understanding-LSTMs/

RNN

❖ Standard RNN + Cell state

- 오랜 state가 경과하더라도 정보가 유실되지 않도록 중요한 정보 흘러가는 컨베이어 벨트
- Cell state 에서 잊어버릴 건 잊어버리고, 새로 추가할 건 추가하자
- Cell state 에서 Hidden state 정보를 얼마나 가져갈 지 정해주자



https://github.com/heartcored98/Standalone-DeepLearning

RNN

❖ advanced RNN: LSTM

Cell state

- 오랜 state가 경과하더라도 정보가 유실되지 않도록 중요한 정보 흘러가는 컨베이어 벨트
- Cell state 에서 잊어버릴 건 잊어버리고, 새로 추가할 건 추가하자 → Forget & Input gate
- Cell state 에서 Hidden state 정보로 얼마나 가져갈 지 정해주자 → Output gate

RNN

❖ Gate

- 문, A gate or gateway is a point of entry to a space which is enclosed by walls. Gates may prevent or control the entry or exit of individuals. (Wikipedia)
  출입을 막거나 통제
- 계수를 곱해주는 연산을 통해 제어 (element wise coefficient multiplication)
- 계수를 0~1사이 값으로 바꾸어 얼마나 잊어버리고, 추가할 것인지 정의

"가중치"

https://github.com/heartcored98/Standalone-DeepLearning

RNN

LSTM network

- Gate
  - 계수를 곱해주는 연산을 통해 제어
  - 계수를 0~1사이 값을 곱해서 얼마나 잊어버리고, 추가할 것인지 정의

현재: t

-0.2	0.1	0.5	0.7	0.9	-0.3	-1.1	2.0	0.6	0.7

C_{t-1}

https://github.com/heartcored98/Standalone-DeepLearning

RNN

LSTM network

❖ Gate

- 계수를 곱해주는 연산을 통해 제어
- 계수를 0~1사이 값을 곱해서 얼마나 잊어버리고, 추가할 것인지 정의

현재: t

0	0.2	0.99	0.1	0.98	0.2	0.1	0.90	0.4	0.1
-0.2	0.1	0.5	0.7	0.9	-0.3	-1.1	2.0	0.6	0.7

$G\ (gate)$

C_{t-1}

https://github.com/heartcored98/Standalone-DeepLearning

RNN

LSTM network

❖ Gate

- 계수를 곱해주는 연산을 통해 제어
- 계수를 0~1사이 값을 곱해서 얼마나 잊어버리고, 추가할 것인지 정의

현재: t

element wise
coefficient multiplication

-0.06	0.02	0.495	0.07	0.882	-0.06	-0.11	1.8	0.24	0.07
0	0.2	0.99	0.1	0.98	0.2	0.1	0.90	0.4	0.1
-0.2	0.1	0.5	0.7	0.9	-0.3	-1.1	2.0	0.6	0.7

$G\ C_{t-1}$

$G\ (gate)$

C_{t-1}

https://github.com/heartcored98/Standalone-DeepLearning

RNN

LSTM network

❖ Gate

- 계수를 곱해주는 연산을 통해 제어
- 계수를 0~1사이 값을 곱해서 얼마나 잊어버리고, 추가할 것인지 정의
- **목적에 따라서 학습하는 linear transformation이 다름**

Linear transformation

$$g_t = \sigma(W_g \cdot v_{input})$$

0~1 input vector

0	0.2	0.99	0.1	0.98	0.2	0.1	0.90	0.4	0.1

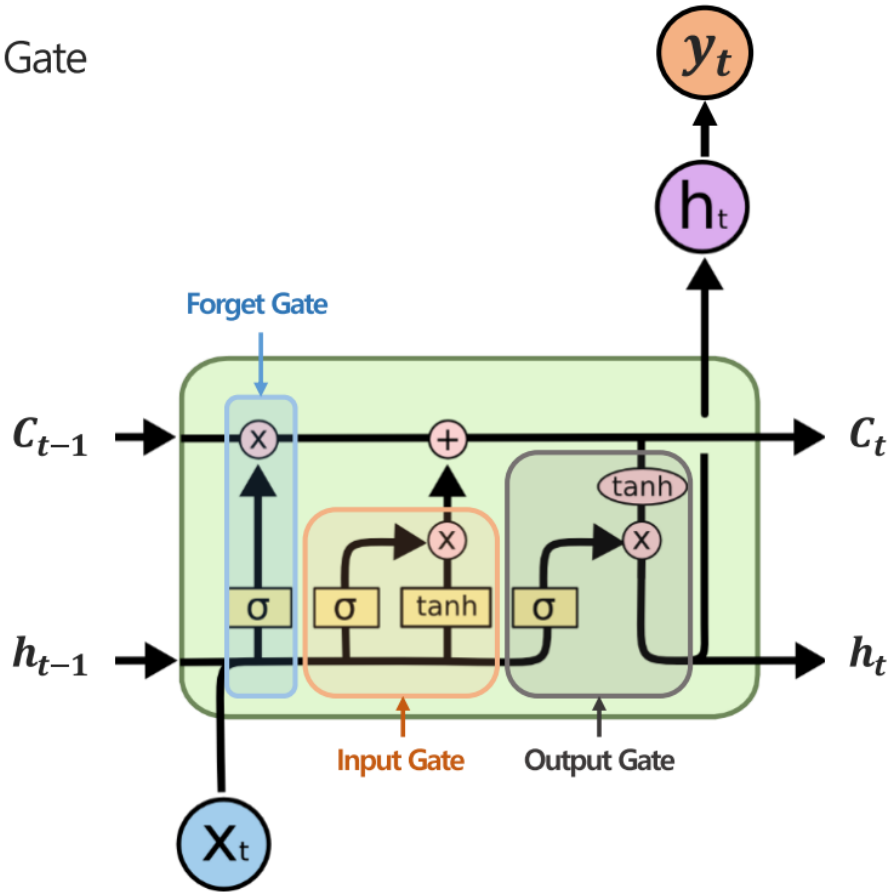
$G\,(gate)$

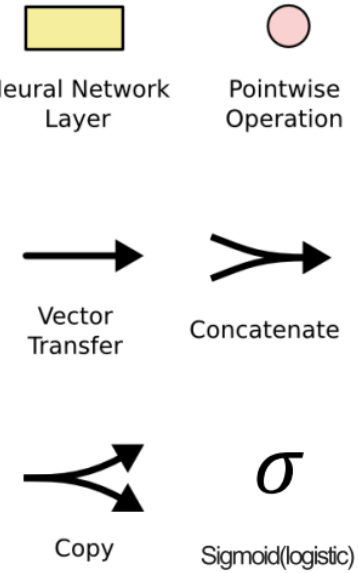
https://github.com/heartcored98/Standalone-DeepLearning

RNN

LSTM network

❖ Gate



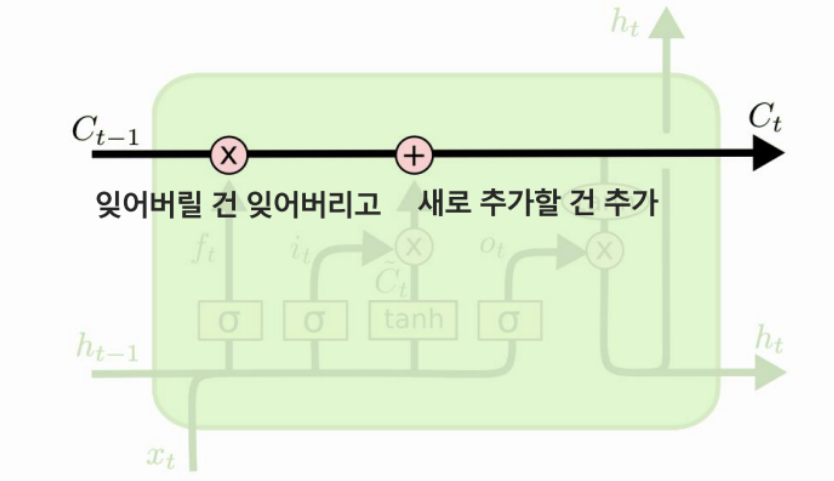


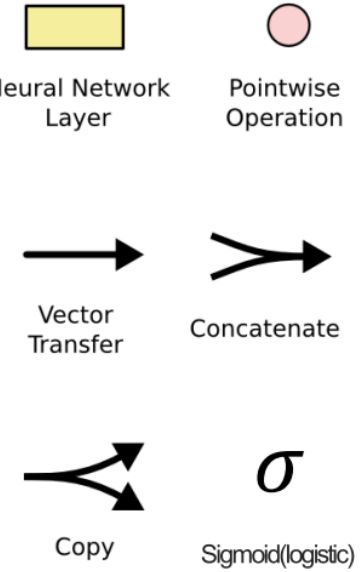
https://github.com/heartcored98/Standalone-DeepLearning

RNN

LSTM network

- Cell state
  - 오랜 state가 경과하더라도 정보가 유실되지 않도록 중요한 정보 흘러가는 컨베이어 벨트
  - 잊어버릴 건 잊어버리고, 새로 추가할 건 추가하자





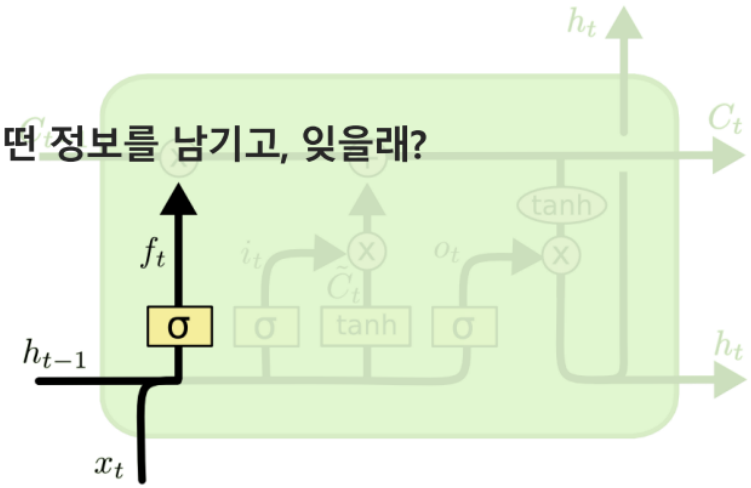
https://colah.github.io/posts/2015-08-Understanding-LSTMs/

RNN

LSTM network

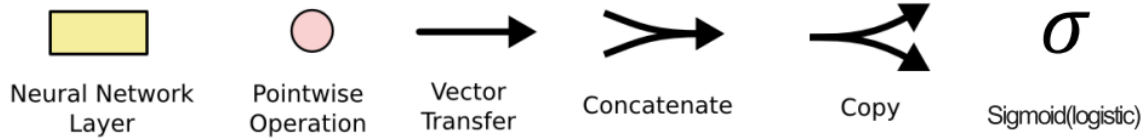
❖ Forget gate: **잊을 것은 잊자**

- Forget gate(f_t): 과거의 cell state 정보 C_{t-1}을 얼마나 받아 들일지 결정



$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

[ ] : concatenate
. : point wise operation



https://colah.github.io/posts/2015-08-Understanding-LSTMs/

RNN

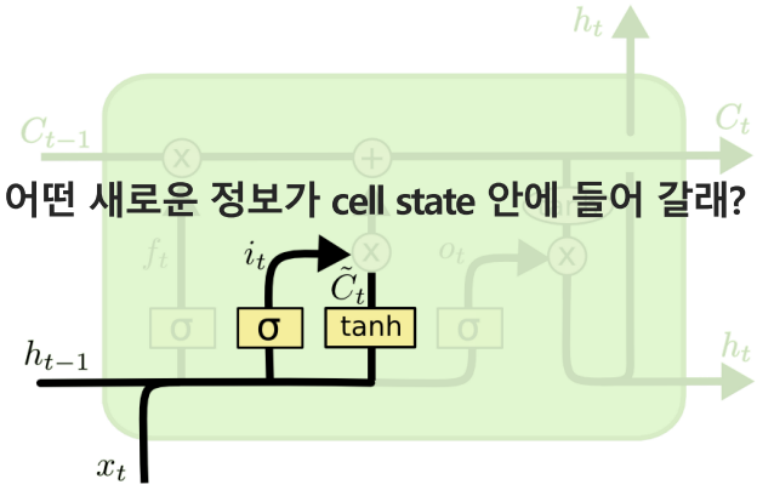
LSTM network

❖ Input gate: **추가할 것은 추가하자**

어느 값을 얼마나 업데이트 할지

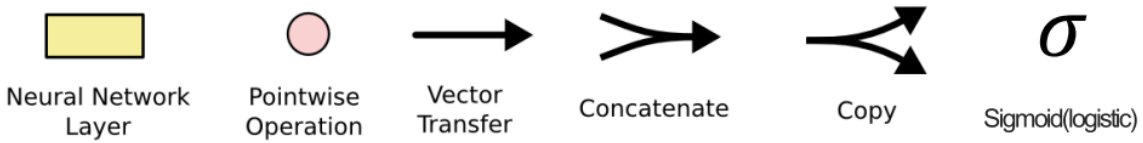
- Input gate(i_t): cell state C_t에 과거 시점의 입력과 출력을 얼마나 반영할 것인지 결정
- Candidate value($\tilde{C}_t$): 과거 시점의 출력(h_{t-1})과 현재 입력 (x_t)정보를 이용해 현재 시점의 C_t 업데이트

어느 값 후보 (Cell state 후보들)



$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$



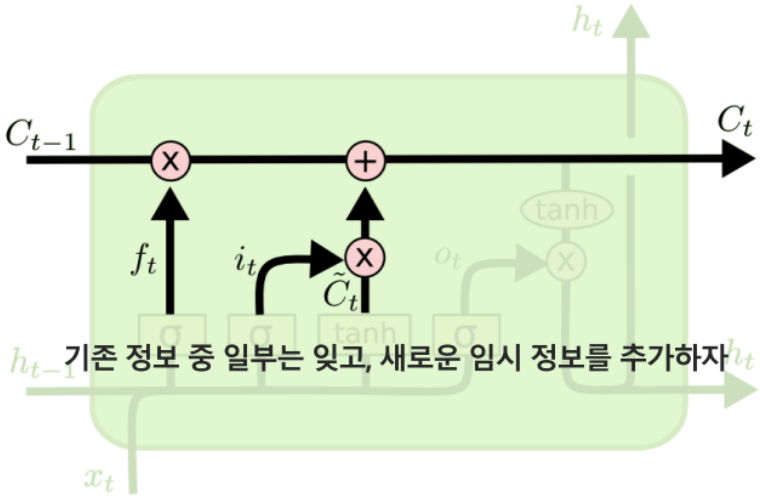
https://colah.github.io/posts/2015-08-Understanding-LSTMs/

RNN

LSTM network

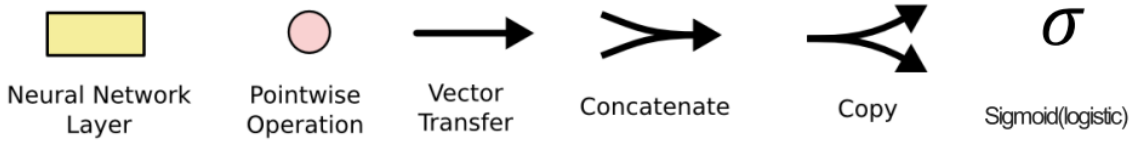
❖ C_t: Cell state update

- 잊을 것은 잊고, : Forget gate (f_t)
- 추가할 것은 추가하자. : Input gate (i_t)



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

1	1	1	1	1	1	1	1	1	1	$f_t(forget\ gate)$
-0.2	0.1	0.5	0.7	0.9	-0.3	-1.1	2.0	0.6	0.7	C_{t-1}



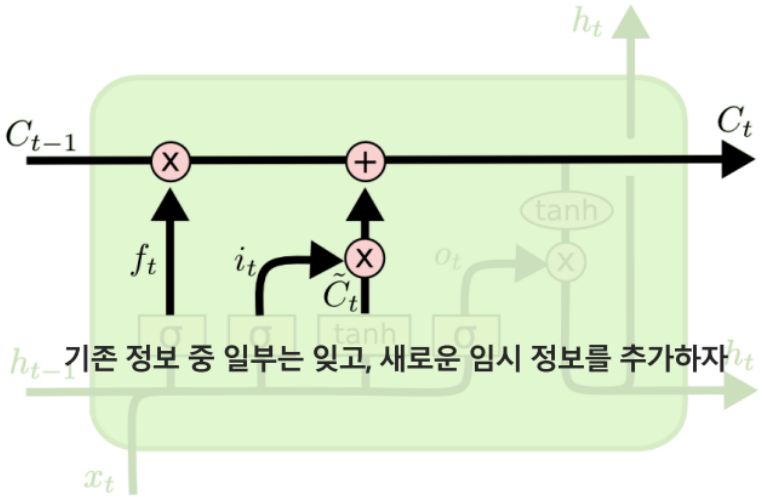
https://colah.github.io/posts/2015-08-Understanding-LSTMs/

RNN

LSTM network

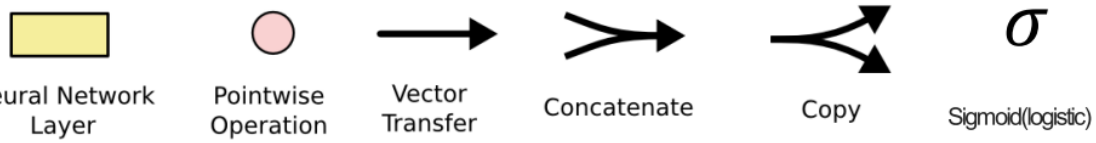
❖ C_t: Cell state update

- 잊을 것은 잊고, : Forget gate (f_t)
- 추가할 것은 추가하자. : Input gate (i_t)



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

0	0	0	0	0	0	0	0	0	0	$f_t(forget\ gate)$
-0.2	0.1	0.5	0.7	0.9	-0.3	-1.1	2.0	0.6	0.7	C_{t-1}



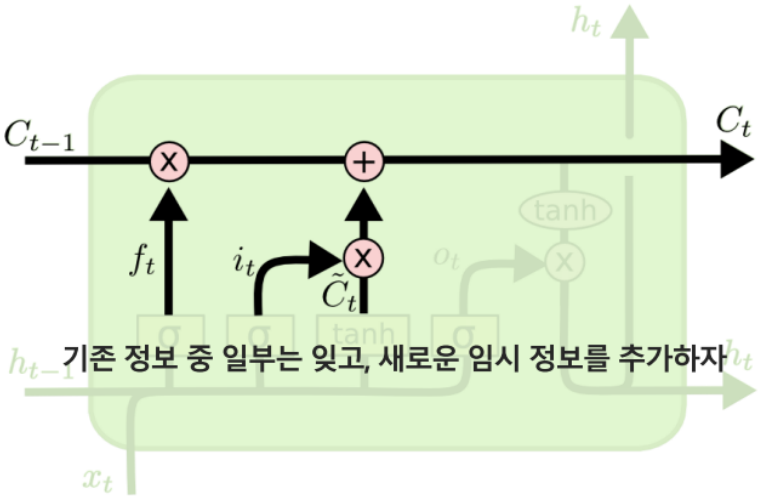
https://colah.github.io/posts/2015-08-Understanding-LSTMs/

RNN

LSTM network

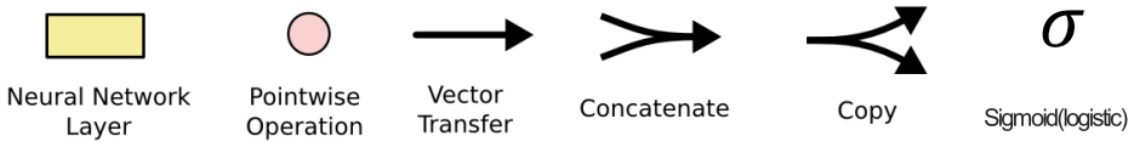
❖ C_t: Cell state update

- 잊을 것은 잊고, : Forget gate (f_t)
- 추가할 것은 추가하자. : Input gate (i_t)



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

0	0.2	0.9	0.1	1	0.2	0.1	0.9	0.4	0.1	$f_t(forget\ gate)$
-0.2	0.1	0.5	0.7	0.9	-0.3	-1.1	2.0	0.6	0.7	C_{t-1}



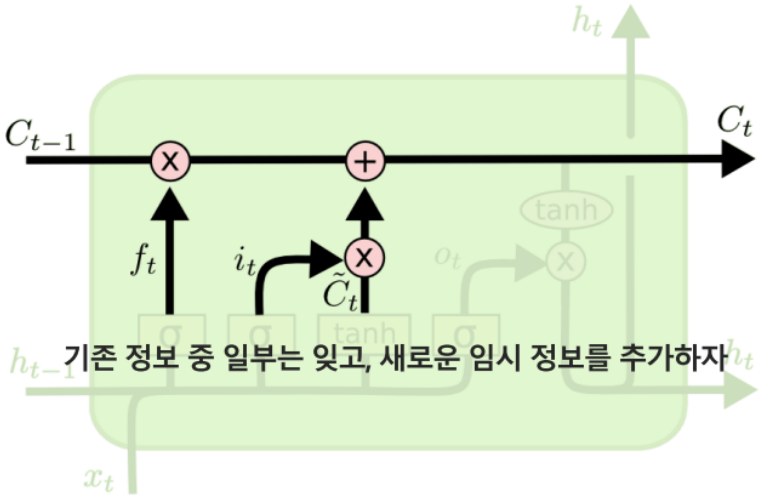
https://colah.github.io/posts/2015-08-Understanding-LSTMs/

RNN

LSTM network

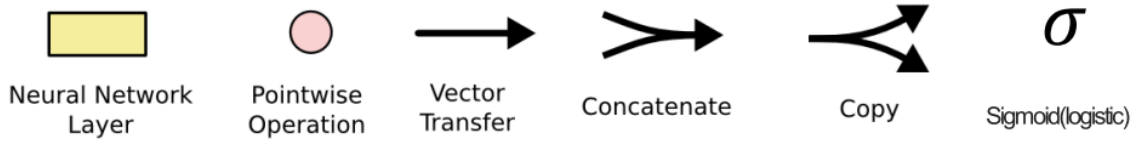
❖ C_t: Cell state update

- 잊을 것은 잊고, : Forget gate (f_t)
- 추가할 것은 추가하자. : Input gate (i_t)



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

0.1	0	0.8	0.2	0.8	0	1	0.9	0.2	0.4	$i_t(input\ gate)$
-0.1	0.3	0.6	0.2	0.9	-0.1	0.4	2.1	0.6	0.9	$\tilde{C}_t$



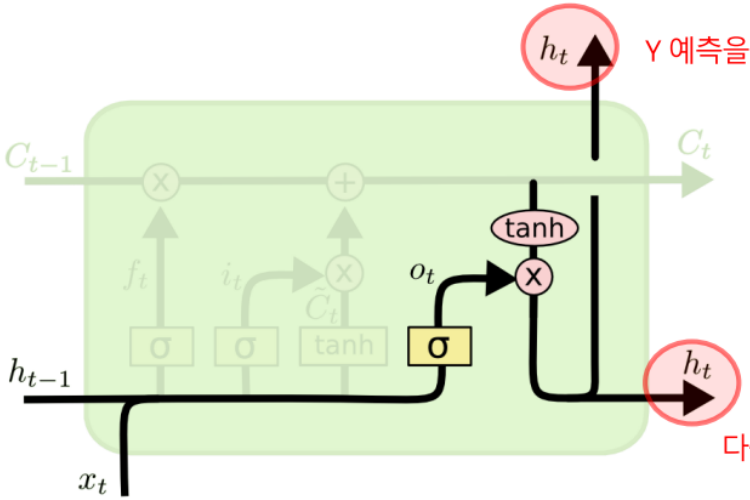
https://colah.github.io/posts/2015-08-Understanding-LSTMs/

RNN

LSTM network

❖ h_t: Hidden state update: **잊을 것 잊고, 추가할 것 추가한 hidden state 전달**

- Output gate(o_t): Cell state에서 Hidden state로 넘겨줄 것은 넘겨주자
- 과거 시점의 출력(h_{t-1})과 현재 입력 (x_t)정보와 C_t를 조합하여 출력 h_t 생산

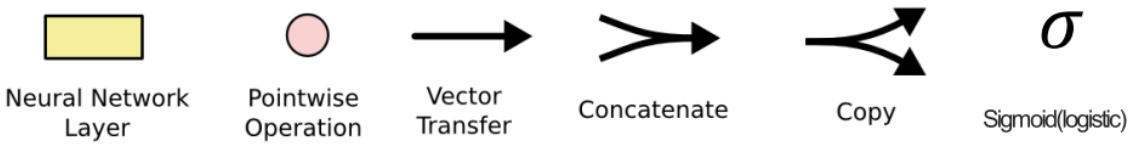


Y 예측을 위한 h_t 내보내기

$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

다음 시점 hidden state 를 위한 hidden state 내보내기

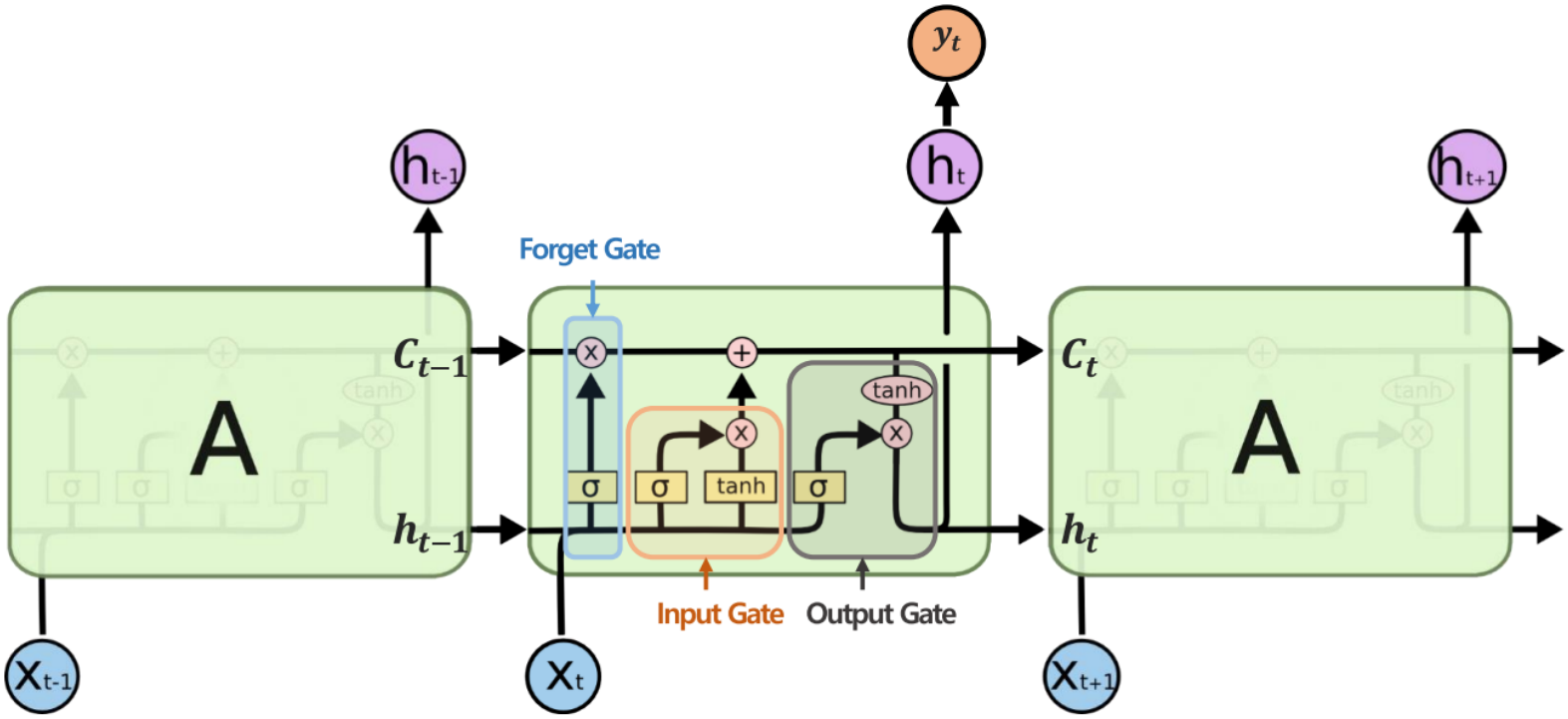


https://colah.github.io/posts/2015-08-Understanding-LSTMs/

RNN

LSTM network

- advanced RNN: LSTM
  - C_t, h_t 두 정보를 모두 제공



RNN

LSTM network

❖ advanced RNN: LSTM

- Work flow

1. Cell state C_{t-1}에서 지울 것은 지운다.
2. 새로운 Cell state 후보 $\tilde{C}_t$에 새로운 input으로 중요한 정보를 넣는다.
3. 추가할 거 추가해서 새로운 Cell state C_t를 만든다.
4. Cell state C_t를 적절히 가공하여 t에서의 hidden state h_t를 만든다.
5. Cell state C_t과 hidden state h_t를 다음 t+1시점으로 전달한다.

$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

$$o_t = \sigma\left(W_o \; [h_{t-1}, x_t] \; + \; b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

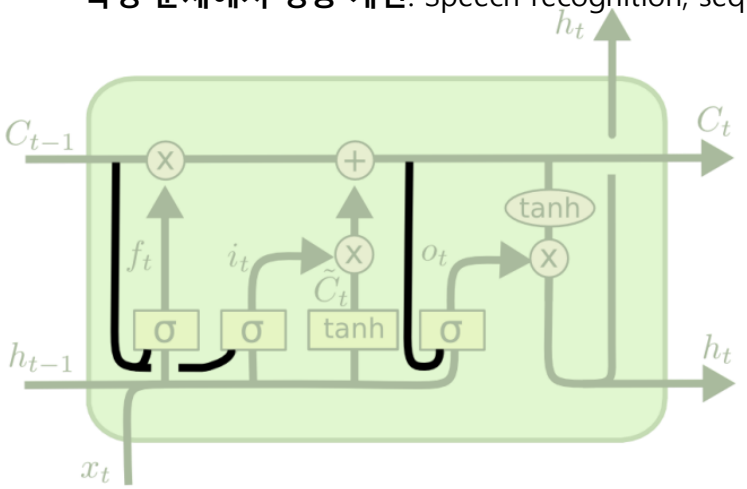
https://colah.github.io/posts/2015-08-Understanding-LSTMs/

RNN

LSTM 패턴 변화

❖ Peephole connections

- **Peephole connections**는 LSTM(Long Short-Term Memory)의 변형 중 하나로, **cell state**의 정보를 각 게이트에 추가적으로 전달하는 메커니즘
- **더 정교한 게이트 조절**: 게이트가 cell state의 정보를 직접 활용하므로, 정보의 흐름을 더 잘 제어
- **복잡한 의존성 학습**: 특히, time step 간 복잡한 의존성을 다룰 때 더 나은 성능
- **특정 문제에서 성능 개선**: Speech recognition, sequence modeling 등에서 peephole connections이 성능을 향상



$$f_t = \sigma\left(W_f \cdot [\boldsymbol{C_{t-1}}, h_{t-1}, x_t] + b_f\right)$$
$$i_t = \sigma\left(W_i \cdot [\boldsymbol{C_{t-1}}, h_{t-1}, x_t] + b_i\right)$$
$$o_t = \sigma\left(W_o \cdot [\boldsymbol{C_t}, h_{t-1}, x_t] + b_o\right)$$

https://colah.github.io/posts/2015-08-Understanding-LSTMs/

RNN

LSTM 패턴 변화

❖ GRU(Gated Recurrent Unit) http://www.kyunghyuncho.me/

조경현 교수

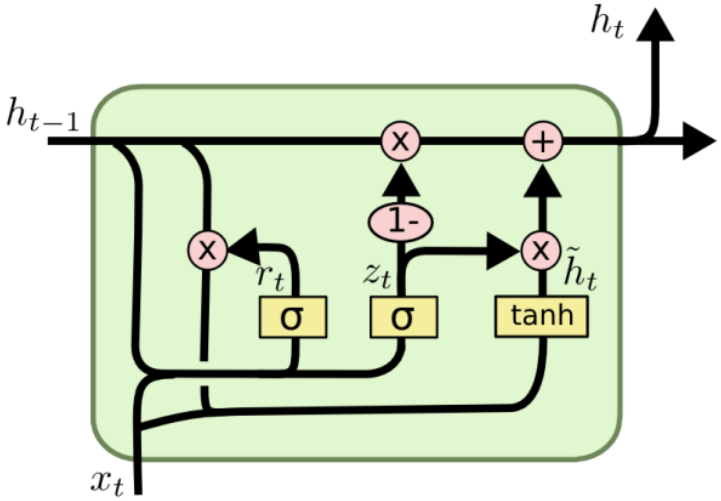
New York University, Facebook AI Research

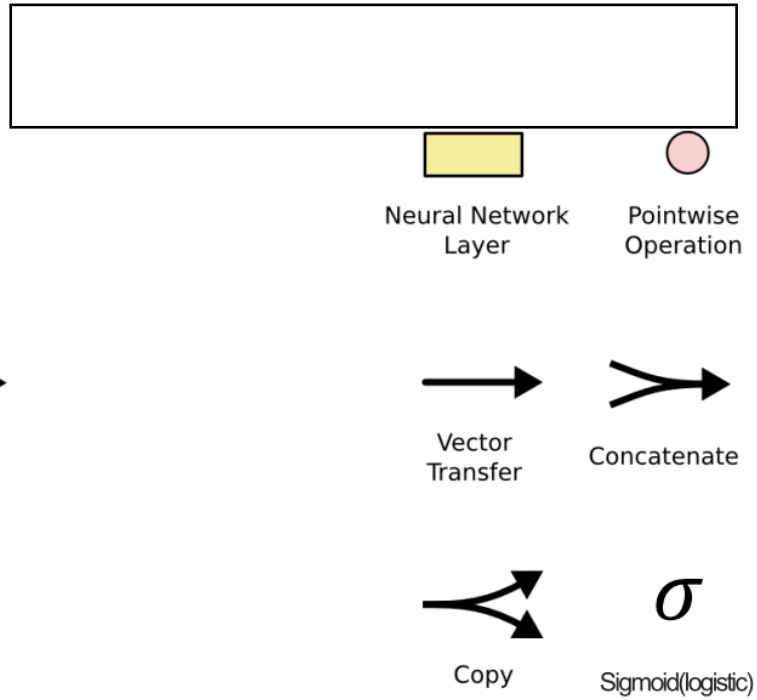
RNN

LSTM 패턴 변화

❖ GRU(Gated Recurrent Unit)

- LSTM의 gate일부를 생략한 형태
- Cell state(C_t)가 다시 없어진 형태
- Reset gate(r_t)와 Update gate(z_t)로 구성





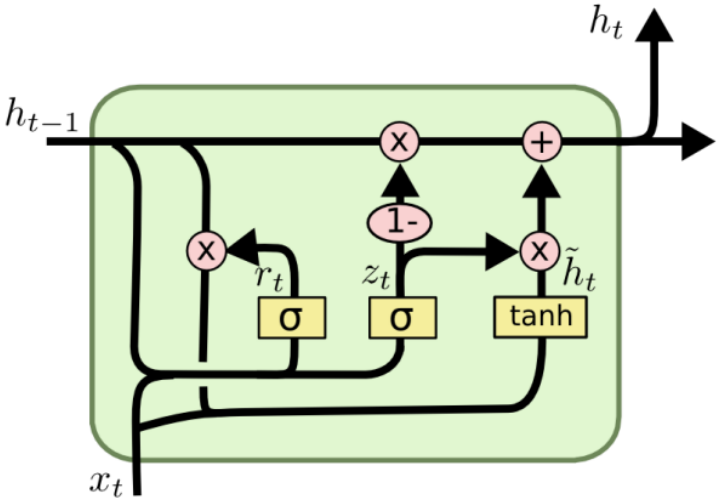
https://colah.github.io/posts/2015-08-Understanding-LSTMs/

RNN

LSTM 패턴 변화

❖ GRU(Gated Recurrent Unit)

- Reset gate(r_t): 잊을 것은 잊자 (~forget gate)
- 임시 output $\tilde{h}_t$를 생성



Reset gate(r_t)

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

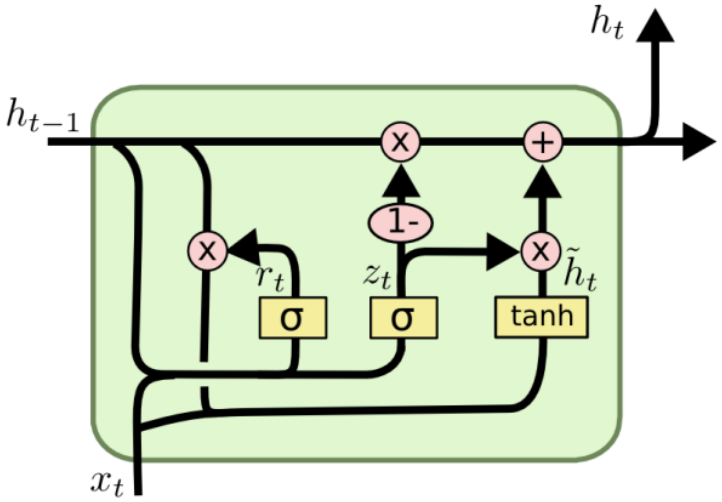
https://colah.github.io/posts/2015-08-Understanding-LSTMs/

RNN

LSTM 패턴 변화

❖ GRU(Gated Recurrent Unit)

- Update gate(z_t): $\tilde{h_t}$와 h_{t-1}간의 비중을 결정
- 최종 output h_t를 생성



Reset gate(r_t)

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

Update gate(z_t)

$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

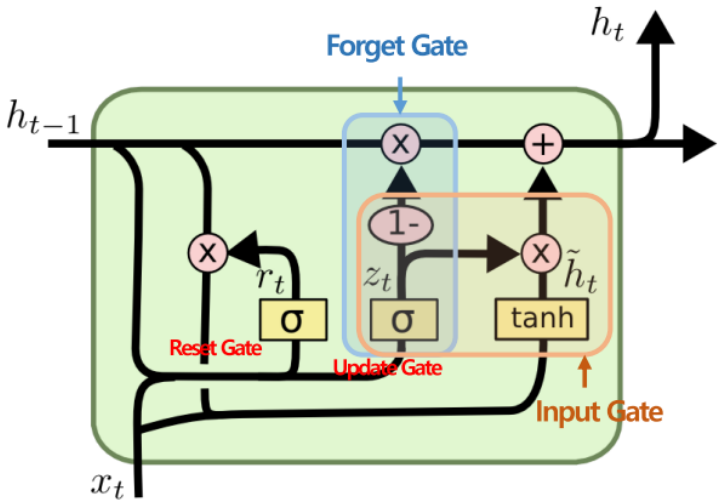
https://colah.github.io/posts/2015-08-Understanding-LSTMs/

RNN

LSTM 패턴 변화

❖ GRU(Gated Recurrent Unit)

- Update gate(z_t): $\tilde{h_t}$와 h_{t-1}간의 비중을 결정
- 최종 output h_t를 생성



Reset gate(r_t)

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

Update gate(z_t) = Forget gate + input gate

$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

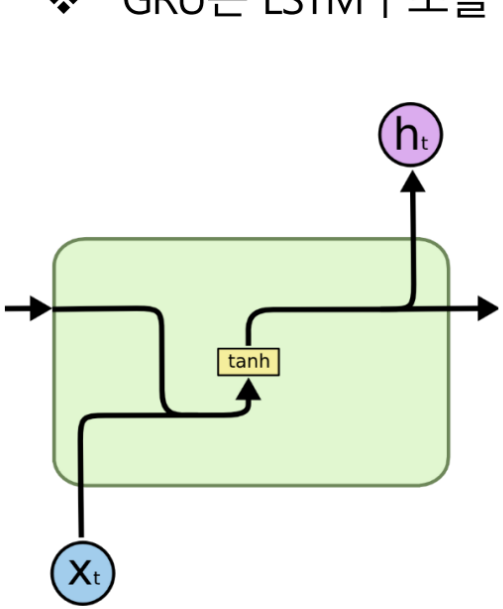
$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

잊을 것은 잊고 추가할 것은 추가하자

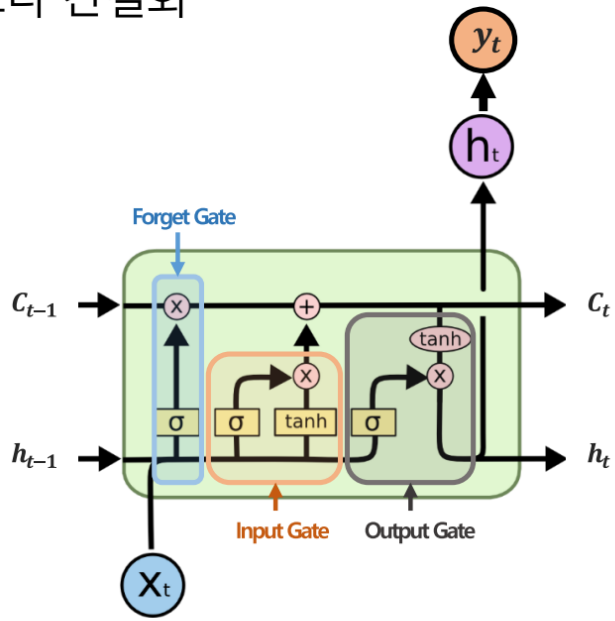
https://colah.github.io/posts/2015-08-Understanding-LSTMs/

RNN

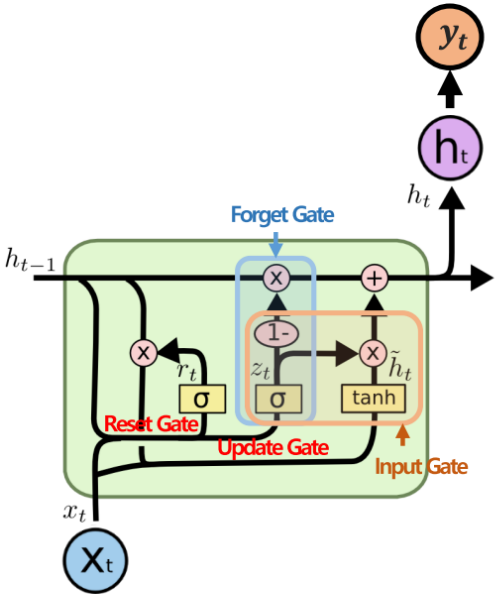
- RNN은 Vanishing gradient(long-term dependency)문제 발생
- LSTM은 Cell state를 만들어서 non-linear 우회하도록
- GRU는 LSTM구조를 보다 간결화



<RNN>



<LSTM>



<GRU>

https://colah.github.io/posts/2015-08-Understanding-LSTMs/

RNN

What else is there?

정답은 없는데 가이드라인을 줄게

1. 천리길도 한걸음 부터! 길이가 짧은 시퀀스에 첫번째 시도 → RNN 사용
2. 길이가 길어, 데이터가 많아? → LSTM 사용
3. 길이가 길어, 데이터가 많아, 시간이 없어? → GRU 사용
4. 나머지는 경험상 논외, 주목해 볼 테크닉은 Attention!